Shanjie: An Open-Source Zhuyin Input Method with Neural Reranking

Nanako Tsai*

Version v1, 2026-11-12

This paper is licensed under the Creative Commons Attribution 4.0 International licence (CC BY 4.0, https://creativecommons.org/licenses/by/4.0/).

Abstract

Zhuyin (Bopomofo) input is a syllable sequence, and the input method must pick, among all homophonous strings, the one the user meant. We present *Shanjie* (善解), an open-source macOS Zhuyin input method, as shipped in v0.7.0 (2026-11-11), and ask what improves its character selection. The system is a layered n-gram decoder followed by a 14.6-million-parameter character language model that may only re-rank the decoder's eight best paths behind a margin gate. Our main finding is that gains came from training text in the right register and from how the re-ranker's score is fused with the decoder's, not from a larger neural component: the largest release step is the model generation rebuilt with forum and chat-channel text, whereas a larger re-ranker, long left context, listwise and distillation objectives and LLM judges all failed their pre-specified criteria, and re-rankers retrained with the same recipe swap 65–100 rows of a chat set with a net within ±12, a noise floor most model changes did not clear. Components were accepted by paired statistics; confirmations, pre-registered since 2026-10-18, run once on unseen data. Of the 13 one-shot confirmations run, 8 held and 5 did not. The largest step, an earlier reading term, lexicon packs and smaller v0.7.0 rules shipped without one, on seen sets only. On the author's 4,958-clause chat development set, seen while values were selected, first-choice accuracy rose from 84.89% to 94.07% in the chat profile between v0.4.0 and v0.7.0 (83.56% to 93.71% in the written profile). On the release-monitoring hold-out, v0.5.0→v0.6.0 moved the public colloquial half from 115 to 124 of 150 (exact McNemar $p = 0.012$; the fifth reported use of that half) and the newly activated news half from 125 to 128 ($p = 0.25$); v0.6.1→v0.7.0 moved them to 127 ($p = 0.25$) and 126 ($p = 0.63$). All evidence comes from one user's chat register and tool-generated readings.

1 Introduction

When a Taiwanese user types Mandarin with Zhuyin, each keystroke group is a syllable with a tone, and almost every syllable maps to many characters. The input method (IME) therefore solves a sequence-labelling problem under a hard constraint: the output must be a string whose reading is exactly the typed syllables, and the system should pick the string the user intended, as often as possible, within a few milliseconds per key. Two complications make the problem harder than dictation-style conversion. First, the same user writes in two registers, casual chat and written prose, and the word choices and frequencies differ enough that one set of parameters serves neither well. Second, the errors that matter to a user are concentrated in homophone pairs that the language

*Repository: https://github.com/Nanako0129/shanjie. Website: https://shanjie.nyanako.com.

itself does not disambiguate locally (在/再, 的/得, 做/作), in proper names, and in colloquialisms absent from edited text.

Shanjie is an open-source (Apache-2.0) macOS IME with one stated priority: the text committed to the application must always be one of the candidates produced by the local lexicon lattice, and selection among those candidates should be as correct as the data allow [35]. This paper describes the system shipped in v0.7.0 (released 2026-11-11) and the evidence gathered in about six weeks of development. It asks three questions.

RQ1 What moves first-choice accuracy in a Zhuyin IME that already has an n-gram decoder: more training text in the user's register, more decoder structure, or a larger neural re-ranker?

RQ2 Under what decision rule can a small neural re-ranker overwrite the decoder's first choice without breaking more than it fixes?

RQ3 How far can results selected on development data be trusted: how much do they shrink on unseen data, and how large is the retraining noise against which a model change must be judged?

Our findings, with the evidence each rests on:

1. **In-register text, not a larger neural model** (RQ1). The largest release step on the chat development set (4,358→4,645 of 4,958 clauses) is the model generation whose bigram and re-ranker were rebuilt with forum and chat-channel text; the rebuilt bigram alone moved the set from 4,366 to 4,534, and the re-ranker retrained on the same text added $+88/-26$ on top (Table 5). A larger re-ranker, long left context, a listwise objective, distillation from a stronger teacher and LLM judges each failed its pre-specified criterion within the conditions tested (Table 6).

2. **How the re-ranker's evidence is fused matters as much as its weights** (RQ2). A colloquial table added as a decoder term broke too many rows once the full pipeline ran; the same table behind a soft gate in the fusion passed its confirmation. For the last re-ranker the written-profile gain is almost entirely the fusion cell and not the weights. The hard margin threshold is not what does the work: at the standard strength, removing it changes none of 2,992 public chat rows, two of 2,182 written rows and one of 4,919 private chat rows (Table 4; Sections 5.3 and 5.4; rows 3, 12 and 13 of Table 1).

3. **Retraining churn is the noise floor** (RQ3). Two re-rankers trained with the same recipe swap 65–100 rows of a chat set with a net within ±12; continuation training lowers the swap to 21–33 rows; and the shipped fusion cell had been selected for one seed (Section 5.4).

4. **Selection sets are optimistic** (RQ3). The broken-to-fixed ratio of a selected cell rose from the selection set to new data every time it was checked, and 5 of the 13 one-shot confirmations did not hold, four of them after passing on data already seen (Table 1).

5. **A conversion table had silently rewritten references and training text**: 127 lexicon words, the root cause of one class of user-reported errors (Section 5.4).

The contributions are (i) a description of the selection pipeline in which every decision rule is an explicit score formula matched to the code (Section 3); (ii) an evaluation protocol for a single-developer IME project, with the full ledger of its one-shot confirmations, held or not (Sections 4 and 5); and (iii) the negative results, classified by the kind of failure, with the criteria they missed (Section 6). Project code names, dates and the complete record of each component are in the appendices.

How the work was done. The implementation, the measurements and the text of this paper were produced by LLM-based coding agents. Throughout the paper an *agent* is one LLM session, and an *independent agent* is a fresh session that did not implement the change under review and receives only a written brief. Independent agents performed code review, acceptance and the hold-out runs; design reviews and research advice were taken from several models. The author takes responsibility for every number and statement in this paper.

2 Related work

We compare prior work with this paper along five lines: the task, the output space, the input given to the model, the evaluation, and the position this paper takes.

Task. Converting a phonetic key sequence into Chinese characters has been treated as statistical language-model decoding since the Pinyin work of Chen and Lee [11] and Gao et al. [16], with joint handling of typing errors [22], and more recently with neural models [20, 42, 54, 58]. Deployed engines add compressed dictionaries and language models [26] or finite-state decoders [19]. For Zhuyin, open-source engines include libchewing [44], RIME [40], McBopomofo [37], whose lexicon is Shanjie's base layer, and vChewing [53]. Shanjie's decoder is of the statistical kind: absolute discounting [10, 36] with a Kneser–Ney continuation distribution [24] (because the training counts are expected counts over a word lattice, the continuation counts are rounded integers with a threshold rather than the fractional-count variant of [57], and the Pitman–Yor generalisation [43] was not pursued), and word classes from an exchange clustering of the class-bigram objective of [8] (cf. [30]).

Output space. In all of these the output is restricted to strings whose reading matches the input; PinyinGPT [42] makes the constraint explicit by restricting the output distribution to homophones. Shanjie is stricter: the neural model never generates, it only reorders the eight best paths of the lexicon lattice. rime-wanxiang and librime-witogram [2, 28] report a negative result that the project adopted as a design rule: replacing collocation tables by raw likelihoods lowered top-1, so Shanjie's re-ranker may overwrite the n-gram first choice only when the fused score clears a margin threshold selected on specified data (Section 3.5). That gate is a selective-prediction rule in the sense of [17], related to learning to defer [34]; we do not claim calibrated probabilities in the sense of [18].

Input. Systems differ in what the neural model sees. The 0c (靈犀注音) IME publishes a Transformer that re-ranks statistical candidates using the last 64 characters of context and the readings, and reports on its own 6,000-sentence news set 76.9% first-choice accuracy for the statistical engine and 86.0% with the Transformer [1, 15] (self-reported, with dictionary-synthesised readings on a different set; not comparable with any number in this paper; §rl-051). The pinyin-ime-reranker-4M model card [31] reports p95 latency falling from 20.64 ms to 8.61 ms when the prefix state is shared across candidates and keystrokes; Zenzai [3] segments its input with reserved tokens, context before readings. Shanjie's re-ranker sees the candidate string and at most two characters of left context, and no readings. Two ideas from this literature were tried and not adopted: a discriminative (listwise) objective motivated by [41] and [45], and data selection by cross-entropy difference [33] (Section 6). Kuhn and De Mori's cache language model [27] is the reference for the planned incremental self-learning.

Evaluation. KySS [21] and the robustness analysis of adaptive Chinese input methods by Jiang et al. [23] motivated the keystroke score of the project's replay tool; AJIMEE-Bench [4] motivated

freezing judging sets into fixed subsets. Our paired tests follow [12] (exact McNemar) and [25] (paired bootstrap); on the practice of significance testing in NLP see [5, 13]. Two strands of the machine-learning literature correspond to what we call fixed and broken rows and churn: regressions and prediction churn introduced by model updates [32, 55, 56] and the variance that seeds and other sources add to benchmark scores [7, 39]. Our layered sets, with single-use judging sets and a use-counted hold-out, are a manual answer to the adaptive-reuse problem studied by [6, 14, 38], and committing the primary judgement before reading a number is preregistration in the sense of [52].

Position of this paper. The task is toned Zhuyin to Traditional Chinese as written in Taiwan; the output space is the paths of a local lexicon lattice; the input is the typed syllables plus at most two characters of left context; the evaluation is paired comparison against the shipped build, with one-shot confirmations. The paper does not propose a new model. It reports, for one open-source system, which changes helped, which did not, and how reliable the measurements are. A day-one snapshot of three existing IMEs, with its limits, is in Appendix F; the literature that changed a design decision is listed in §ip-008.

3 The selection pipeline

Figure 1 shows the flow from a key to the screen; the subsections below define each box.

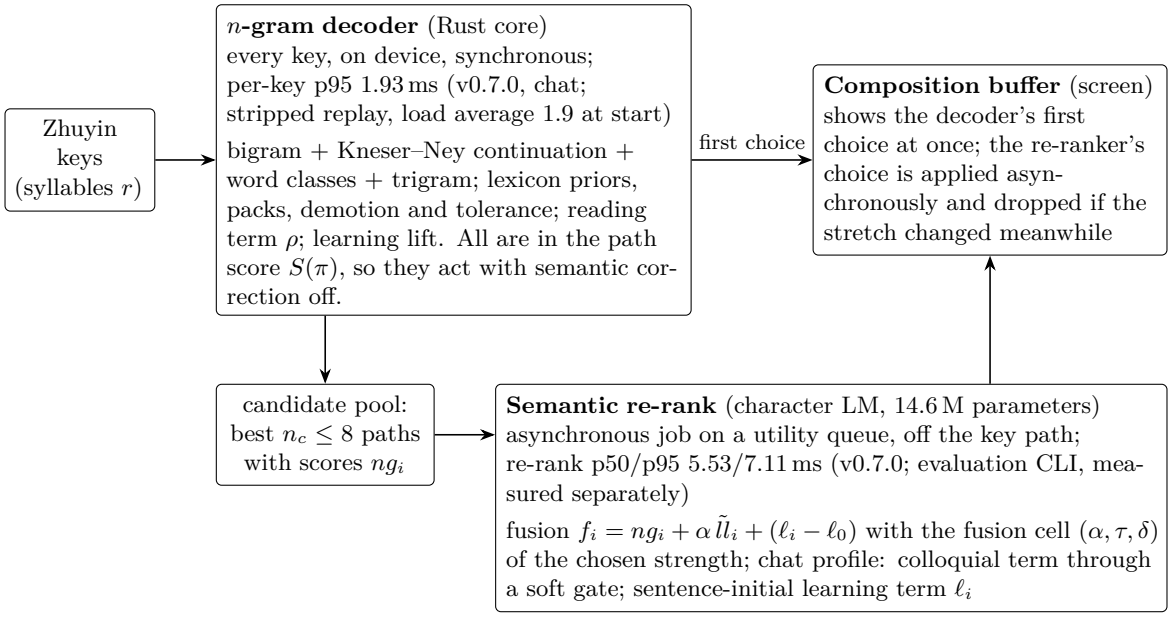


Figure 1: The selection pipeline. The decoder runs on every key and its first choice is displayed at once; the re-ranker only reorders the candidate pool in the background and never generates text. The terms in the decoder box act in both profiles, at every strength and with semantic correction off; the colloquial term and the strength triple (α, τ, δ) act only in the re-rank. The per-key figure is a stripped replay of the synchronous key path (the decoder's model files; no packs, no left context, no semantic correction), not full-application latency (Section 3.8).

This section describes the pipeline shipped in v0.7.0. The release before it, v0.6.1, differs from this description in three larger places: an earlier set of re-ranker weights and fusion cells, a reading term placed inside the re-rank fusion rather than in the decoder (Appendix A.7), and neither the

sentence-initial estimate of the colloquial term (Section 3.6) nor the sentence-initial learning rule (Appendix A.8). The re-ranker weights, the fusion cells, the decoder reading term and the learning rule each have a one-shot confirmation (rows 11, 12–13, 6 and 9 of Table 1); the sentence-initial estimate of the colloquial term has none of its own: the judgement of row 11 is in the written profile, where the term does not act, and both arms of the chat-profile judgements of rows 12 and 13 carried it (§rl-110, §rl-113, §rl-114). Three smaller additions of v0.7.0, which have no one-shot confirmation either, are marked where they are described: a rule for tied readings in the reading term (Appendix A.7), a five-word protect list in the re-rank mask (Section 3.5) and a third default pack (Appendix A). The release-level numbers of v0.7.0 are the last rows of Tables 2 and 3, and Table 4 takes its system apart.

3.1 Architecture and constraints

The engine is a Rust core behind a C ABI, driven by a Swift InputMethodKit shell. The core builds a lattice of lexicon entries whose readings match the typed syllables, decodes it with the n-gram model, and hands the best path to the shell; the shell shows it in the composition buffer and schedules an asynchronous re-rank job on a utility queue. The re-ranker can only reorder the first eight decoder paths; it never generates text. Left context is read from the client application (at most 256 UTF-16 units, cut at the last line break, capped at 64 grapheme clusters, same message across punctuation); the decoder and the learning store use only the trailing two Han characters of it, and the re-ranker uses the length its model file declares (two characters for every shipped model). The shell chooses one of two profiles, *chat* or *written* (the CLI flag spells the latter `formal`), by the foreground application's bundle identifier; the user may pin a profile globally or per application.

Acceptance targets fixed at the start of the project (§plan-003) were: candidate coverage (oracle@64) $\geq 98\%$ on the hold-out set, per-key p95 latency < 16 ms, re-rank p95 < 300 ms and asynchronous, and the rule that learning from one re-selection must not pollute other sentences.

Lexicon, notation and learning. The lexicon, the priors $lp(w)$ and penalties $\eta(w)$ derived from it, the rules by which a re-selection is learned, and a table of every symbol are in Appendix A. The main text needs three facts only. Every lexicon entry w is a (reading, string) pair with a $\log_{10}$ prior $lp(w)$. The prior after learning is $lp'(w)$, never below $lp(w)$. And $\eta(w) \geq 0$ collects the demotion and tolerance penalties.

3.2 Decoding objective

Let $r = r_1 \cdots r_n$ be the typed syllables (each with its tone). A *path* $\pi = (w_1, \ldots, w_m)$ is a sequence of lexicon entries whose readings, concatenated, equal r; entry w_j covers $l_j \geq 1$ syllables and $\sum_j l_j = n$. $\Pi(r)$ is the set of such paths; $y(\pi)$ is the string $w_1 \cdots w_m$. Each entry carries a reading-specific lexicon prior $lp(w)$ in $\log_{10}$ (Appendix A.3). The profile $p \in \{\text{chat}, \text{written}\}$ fixes the interpolation weight λ_p ($\lambda_{\text{chat}} = 0.5$, $\lambda_{\text{written}} = 0.7$), the Kneser–Ney weight β_p and the trigram table T_p in use. The decoder searches for the path with the highest score,

$$\hat{\pi} \approx \arg\max_{\pi \in \Pi(r)} S(\pi), \qquad S(\pi) = \sum_{j=1}^{m} \Big[\lambda_p \log_{10} P(w_j \mid h_j) + (1 - \lambda_p)\, lp'(w_j) - \eta(w_j) + \rho(w_j) \Big] + E(\pi), \tag{1}$$

where h_j is the history of w_j (the previous word v, and the word before it u when a trigram table is on; h_1 is the fixed word to the left of the stretch or $\langle s \rangle$); lp' is the prior after learning (Appendix A.8); $\eta(w_j) \geq 0$ collects the demotion and tolerance penalties and $\rho(w_j)$ the reading term

(Appendices A.3 and A.7); and $E(\pi)$ is the closing term: $\lambda_p \log_{10} P(\langle/s\rangle \mid h_{m+1})$ when the stretch ends the sentence, and the full word term of the fixed word w_{next} to its right, $\lambda_p \log_{10} P(w_{\text{next}} \mid h_{m+1}) + (1-\lambda_p)\, lp(w_{\text{next}}) - \eta(w_{\text{next}})$, when the user has fixed one. The maximisation is approximate: the search is a beam over syllable positions, so $\hat{\pi}$ is the best path found, not a guaranteed maximiser. At each position one hypothesis is kept per key (surface string, last word), with the previous word added to the key when a trigram table is on, and the best $B = 64$ hypotheses survive. For a reading only its 12 best entries are tried (`PER_KEY`), plus any lower-ranked entry the user has learned under the current context. All scores are $\log_{10}$ and the floating-point operation order is fixed: the Rust core's output is byte-identical to the Python reference on the golden files, and candidate scores agree to 10^{-9} (§plan-009).

3.3 Bigram with absolute discounting

The bigram model stores, for every context word v with kept entries, the expected count $c(v, w)$ of each kept successor (rounded; expected because training sentences are counted over their word lattices), the pre-pruning total $t(v)$ and a discount D (0.75 in the shipped model). For w in the model's vocabulary,

$$P(w \mid v) = \begin{cases} \dfrac{c(v,w) - D}{t(v)} + \text{back}(v)\, pb(w) & c(v,w) > 0, \\ \text{back}(v)\, q(w \mid v) & \text{otherwise}, \end{cases} \qquad \text{back}(v) = 1 - \sum_{w' :\, c(v,w') > 0} \frac{c(v,w') - D}{t(v)}, \tag{2}$$

with $\text{back}(v) = 1$ for a context without entries. $pb(w)$ is the unigram back-off value of w, $10^{lp(w)}$ without a side file (so the lexicon prior, not a corpus unigram, is the back-off distribution), and $q(w \mid v)$ is the class-smoothed back-off of Appendix A.5. Bigrams with weighted count below the pruning threshold (6 in the shipped model) are not kept, but $t(v)$ keeps the pre-pruning total, so pruning moves mass into $\text{back}(v)$.

3.4 Back-off modifiers, trigram table and reading term

Three files modify this bigram and one further term is added to the path score. Their formulas are in Appendix A; in words:

- *Kneser–Ney continuation value* (Equation (8)). The back-off value $pb(w)$ is replaced by a mixture, with weight β_p (1 in chat, 0.75 in written), of the normalised number of distinct left contexts of w and the lexicon prior.
- *Word-class back-off* (Equation (9)). Where the bigram has no kept entry, the back-off value is interpolated, with weight $\mu = 0.2$, with a class bigram over $K = 512$ exchange-clustered classes times a class emission. The term never touches a kept bigram.
- *Trigram table* (Equation (10)). Where the context (u, v) is stored, an absolute-discounted trigram backs off to the bigram and is floored at $F = 0.5$ times the bigram score. The tables (7,700,964 entries in the written profile, 950,000 in chat) are memory-mapped.
- *Reading term* $\rho(w)$ (Equation (11)). A single-character entry used under a reading that is not the character's best-scored reading is penalised by $\lambda_r = 2.0$ times the $\log_{10}$ score difference (the difference capped at 4), with exemptions for tone-sandhi readings and for twelve characters whose dictionary-first reading the lexicon lists second, and with a re-estimate from multi-character words for readings that sit at the lexicon's floor score and, from v0.7.0, for readings

the lexicon scores equal. The term is in the path score, so the beam prunes on it, in both profiles, at every strength and with semantic correction off.

Scores, not probabilities. The quantities written $P(\cdot \mid \cdot)$ below are scores. The bigram of Equation (2) with a lexicon-prior back-off, the substitutions of Equations (8) and (9), and the floor of Equation (10) are not renormalised, so their values over w need not sum to one; pb, pb_{KN} and q are back-off values in the same sense. They are used only to rank paths, and the engine was not changed to make them normalise.

3.5 Semantic correction: the small character LM and the fusion rule

The re-ranker is a six-layer pre-LN Transformer decoder, width 384, six heads, a positional window of 64 characters, tied embeddings, learned positions, GELU, with a character vocabulary of 10,351 ids; 14,646,912 parameters stored as per-row symmetric int8 with a float32 scale, in a 14,886,880-byte file. The 64 is the model's window; the context it is given, the key k, is at most two characters. Let $y_0, \ldots, y_{n_c-1}$ ($n_c \le 8$) be the decoder's best paths of one stretch in decoder order, $ng_i = S(\pi_i)$ their path scores, and k the last at most two Han characters before the stretch (empty when none). The model's score of candidate i is

$$ll_i = \sum_{t=1}^{|y_i|} \log_{10} p_\theta\big(y_{i,t} \mid [\mathrm{BOS}], k, y_{i,<t}\big), \tag{3}$$

all n_c candidates right-padded into one forward pass. A stretch with a single candidate is not re-ranked. Let U_0 be the usable candidates: candidate 0 and every $i \ge 1$ not masked (a candidate is masked when it removes a manually added word that the decoder's first choice contains, or, from v0.7.0, one of the five common words of a rerank-protect list [51]; the fuzzy-input rule of Appendix A.3 also removes candidates containing a tolerance-matched word, and a stretch whose first choice contains one is not re-ranked at all). With the strength's triple (α, τ, δ) and the learning terms ℓ_i of Appendix A.8 ($\ell_0 = 0$ by construction), the decision ("pick") is

$$\begin{aligned} f_i &= ng_i + \alpha\,\tilde{ll}_i + (\ell_i - \ell_0), \qquad U = \begin{cases} \{0\} \cup \{i \in U_0 : \ell_i > \ell_0\} & ng_0 - ng_1 \ge \tau, \\ U_0 & \text{otherwise,} \end{cases} \\ j &= \arg\max_{i \in U} f_i, \qquad \text{choose } \begin{cases} 0 & U = \{0\}, \\ j & f_j - f_0 \ge \delta, \\ 0 & \text{otherwise,} \end{cases} \end{aligned} \tag{4}$$

where $\tilde{ll}_i = ll_i$ in the written profile and the colloquially fused value of Equation (6) in chat. Ties in the argmax go to the better decoder rank, so with $\delta = 0$ a candidate must score strictly above f_0 to replace the first choice. With $\ell \equiv 0$ this is: the decoder's first choice stands when its margin over the second is at least τ; otherwise the best fused candidate replaces it only if it wins by at least δ. The three strengths are three triples and are the only thing the strength setting changes. In v0.7.0 they are, for the conservative, standard and aggressive strengths, chat $(1.3, 2, 1)$, $(1.1, 2, 0)$, $(1.1, 2, 0)$ and written $(0.8, 2, 1.5)$, $(0.8, 2, 0)$, $(0.8, 2, 0)$ (those of v0.6.1 are in Appendix H). The thresholds τ and δ are score margins selected on specified tuning and judging data (Sections 4 and 5.1); they are not calibrated probabilities. The chat triples live in the core (they were selected with the colloquial term on); the written triples live in the model file's header. The shell applies the chosen candidate asynchronously and discards a result whose stretch has changed meanwhile;

a stretch whose decoder first choice contains a learned-lifted word is never submitted (learning guard).

The model file records the SHA-256 of the bigram and the Kneser–Ney file it was tuned with and refuses to load against any other, falling back to the decoder alone. Training text is Wikipedia (first 200,000 articles), Common Voice zh-TW, Tatoeba, synthetic colloquial sentences, g0v `#rand0m`, Forum A (posts and replies from a public Taiwanese forum), and OpenSubtitles v2018 Traditional-Chinese subtitles [29] (4,432,866 sentences, 42,814,742 Han characters after cleaning). The weights were trained from scratch (36,005 steps, one epoch, AdamW, cosine schedule, peak 1e-3, 200 warm-up steps) and then continued for 3,000 steps at peak 1e-4 on the same sources, re-converted without the `TWPhrases` conversion table (Section 5.4).

3.6 Colloquial term in the chat profile

The colloquial table is a plain bigram table (vocabulary $V_{\mathrm{CL}} = 69{,}457$, total N_{CL}) counted on the colloquial pool alone and read without classes or side file, so its scores P_{CL} follow Equation (2) with the table's own discount and the back-off value $pb_{\mathrm{CL}}(w) = (c_{\mathrm{CL}}(w) + 1)/(N_{\mathrm{CL}} + V_{\mathrm{CL}} + 1)$ for words (0 for a word outside the table); the end term backs off to the table's own end-of-sentence rate. For candidate i with word split $w_1 \cdots w_m$,

$$lc_i = \sum_{j=1}^{m} \log_{10} P_{\mathrm{CL}}(w_j \mid w_{j-1}) + \log_{10} P_{\mathrm{CL}}(\langle/s\rangle \mid w_m), \qquad col_i = lc_i - \kappa\, o_i, \qquad \kappa = 0.5, \tag{5}$$

with $w_0 = \langle s\rangle$ and o_i the number of words of the split that come from the overlay or a pack and are absent from the base lexicon. In the shipping setting (the "mild" variant) the first word's factor is $pb_{\mathrm{CL}}(w_1)$ instead of $P_{\mathrm{CL}}(w_1 \mid \langle s\rangle)$ when the table has no $\langle s\rangle$ bigram for w_1. The term enters the fusion relative to the decoder's first choice through a soft gate, with $\sigma(x) = 1/(1 + e^{-x})$:

$$\tilde{ll}_i = ll_i + g_i \,\frac{\mu_{\mathrm{CL}}}{\alpha}\,(col_i - col_0), \qquad g_i = \sigma\Big(\frac{ll_i - ll_0}{T_g}\Big), \qquad \mu_{\mathrm{CL}} = 0.5,\ T_g = 1.0, \tag{6}$$

so that in $f_i = ng_i + \alpha\tilde{ll}_i$ the term's weight is $\mu_{\mathrm{CL}}\, g_i(col_i - col_0)$ whatever α is, and the overlay penalty's weight is $\gamma = \kappa\mu_{\mathrm{CL}} = 0.25$ per extra pack word. The gate reads ll_0: a candidate the character LM itself scores below the anchor gets little of the colloquial evidence. If any col_i is not finite the term is dropped for that stretch and the plain rule with the file header's chat triple is used. The term is active in the chat profile at all three strengths, and only when the loaded re-ranker is the one the cells were measured with (paired by hash).

3.7 Learning from re-selection

When the user re-selects a word, the engine stores a (context key, reading, word) record with a decaying weight and lifts that entry's prior to $lp'(w)$ in later decodes; a lifted entry takes no reading term, and a stretch whose first choice contains one is not sent to the re-ranker. Sentence-initial single characters are handled in the fusion instead, through the term ℓ_i of Equation (4): a taught character is added to the candidate pool and receives a bonus that depends on how much of the following syllables matches the taught context. The formulas are in Appendix A.8; the rule passed its one-shot confirmation at the third version (rows 7–9 of Table 1).

3.8 Latency and memory

Two kinds of latency figure appear in this paper, and neither is full-application latency. The *per-key* figure comes from the cross-version benchmark, which replays 8,336 keys of the 302-sentence development set through the synchronous key path with the model files the engine's loader reads for that release (the bigram and class files and, where the release has them, the Kneser–Ney side file, the trigram tables and the decoder reading term) but no packs, no left context and no semantic correction [51]. It measures the stripped decoder. The *re-rank* figure is measured separately by the evaluation CLI. Neither includes the shell, the candidate window or the application.

On the development Mac, v0.6.0 gave a per-key p95 of 2.232 ms (chat), engine load 945 ms, RSS 480 MB and re-rank p50/p95 6.68/8.58 ms; the release notes report v0.5.0→v0.6.0 as 2.06→2.23 ms, 892→945 ms, 441→480 MB and 8.3→8.6 ms. The v0.6.0 run started at a load average of 5.8 and is therefore not an idle-window measurement. The same replay under heavier load gave 5–8 ms for earlier versions, so the "about 2 ms" figures describe a lightly loaded machine only. The decoder reading term added at most 7.5 MiB and −0.01 to 0.0 ms of p95, measured in an idle window (§rl-089).

For v0.7.0 the benchmark replayed v0.6.1 and v0.7.0 in alternation in one session (three rounds each, the median p95 taken) on a different Mac from the sessions above, so its figures compare with each other and not with those of v0.6.0. The load average was 1.9 when the session started, within the limit of 2.0 under which the project reports a speed figure, and 4.2 when it ended; the speed run comes last, so the end value includes it [51]. From v0.6.1 to v0.7.0 the per-key p95 went 1.91→1.93 ms in the chat profile and 1.92→1.96 ms in the written profile, engine load 0.87→1.05 s, RSS 480→489 MB and re-rank p50/p95 5.56/7.15→5.53/7.11 ms [47, 50].

4 Evaluation methodology

A few terms recur below. An *experiment* is one unit of work with its own written contract or registration; a *cell* is one point of a parameter grid, for the re-ranker a triple (α, τ, δ); a *one-shot confirmation* is a single run on data the experiment has not seen, after which the set is retired; *fixed* and *broken* are the rows a change turns right and wrong against the baseline, *net* is fixed minus broken, the *ratio* is broken divided by fixed, and *churn* is the number of rows two otherwise equivalent systems swap; $U_2 = \text{fixed} - 2 \times \text{broken}$ is the score used to choose between re-ranker trainings. An *independent agent* is an LLM session that did not implement the change under review (Introduction).

4.1 Metrics

The primary metric is sentence-level first-choice accuracy (top-1) under a lenient comparison that maps 她妳它牠嘗周臺裏 to 他你他他嚐週台裡 and treats 2,072 dictionary-listed "also written" variants with the same reading as identical. This is a post-hoc rule, adopted at the start of the project after reading errors on the development set, but it is fixed to the dictionary rather than to the test sentences (§me-001). Secondary metrics are oracle@64 (the correct string is among the first 64 candidates), character error rate (Levenshtein distance of the first choice to the reference divided by reference length), log-loss of the reference under a softmax of the 64 candidate scores, and latency. A "legal forms" score that also accepts dictionary-attested alternative characters is reported but never used for decisions. In tables, "a/b" after a count is fixed/broken.

4.2 Sets and their roles

Table 8 (Appendix B) lists the sets. Three properties organise them. (i) Sentences are clauses cut at punctuation, with the preceding clause of the same message as context. (ii) Readings are produced by a tool from the reference text (single characters take the primary reading), except for the user-reported set, where the readings are what the user actually typed; this difference matters (Section 8). (iii) Each set has one role at a time; the roles below are those in force since the layering of 2026-10-18, and two changed before that: discordtune was the main tuning set until it became the development-layer gate, and dev302 moved from a regression gate in both profiles to a written-only gate. The regression layer (dev302, typing76, probe, function words, wikitypo, user-reported) must be row-identical or lose at most 2 rows with non-negative net; the tuning layer (cvtune-clean, wikitune-clean, chatclause/`ctune`) is where values are selected; the development layer (discordtune/`cdev`, lytune, newstune) is where non-inferiority is checked; the judging layer (chatjudge-k, wikijudge-k, `cjudge`-k) is used once per decision and then retired; and the hold-out (150 public colloquial clauses and 150 private news clauses) is run only by an independent agent before a release tag, with its use count recorded and a rotation to the next shard after eight uses (§me-002, §me-014; [48]).

What the hold-out is. The hold-out is a release-monitoring set, not a never-seen test. Its colloquial half (150 public clauses, chat profile) was active from 2026-10-16 to 2026-11-11, and a statistic computed on it was reported back eight times: four times while it was part of a combined 300-row shard, then at the v0.6.0 release (its fifth use), at the v0.6.1 release (its sixth) and twice for v0.7.0 (its seventh and eighth: the release was measured again after the rerank-protect list had been added, with the same summary line). The rows themselves are never read by the implementing agents and only the summary line is returned; a half is replaced by the next shard after eight uses, or as soon as any decision is made because of its number, and on 2026-11-11 the colloquial half, at eight uses, was replaced by a shard that has not been used. The news half (150 private clauses, written profile) was newly activated for the v0.6.0 release, which was its first use; v0.6.1 was its second and the two v0.7.0 runs its third and fourth. The original 227-row hold-out was read at every release up to v0.4.6 and in several component experiments (Appendix C); the public record gives no use count for it, and it is now a readable challenge set [48]. We therefore cite hold-out numbers as monitoring evidence and take confirmatory evidence only from the one-shot confirmations of Table 1.

4.3 Paired statistics and what was judged

Comparisons are row-paired against a baseline, and we report fixed, broken and net. Two tests are used: the exact two-sided McNemar test on the discordant rows [12], whose sampling unit is the clause, and a paired bootstrap interval for Δtop-1 [25], 95% for reporting and 90% for non-inferiority.

The primary judgement, by period. Which statistic decided an experiment changed once, and we state it for each period.

- *Before 2026-10-18.* Experiments had written contracts, usually a net gain with exact McNemar $p < 0.05$ on a named set together with guards (a cap on the broken rate, no significant loss on other sets). The primary judgement was not committed before measurement, the bootstrap resampled clauses, and some criteria were revised after results (Appendix E). We treat every p value of this period as descriptive.

- *From 2026-10-18.* A confirmation commits one primary judgement before any number is read. For a claim of *improvement* the rule is: net > 0, exact two-sided McNemar $p < 0.05$ on the whole judging set, and ratio ≤ 0.65. For a claim of *no harm* the rule is a non-inferiority margin: the lower bound of the 90% paired bootstrap interval of Δtop-1 must be at least -1.0 percentage point. From this date the bootstrap resamples clusters: all clauses cut from one message, speech or article are drawn together. The records give each set's number of clauses but not its number of clusters.

- *Broken-rate caps* use the one-sided 95% exact Poisson upper bound of the broken count divided by n, not the point estimate, after the point estimate twice selected values that sat at the limit on tuning data and exceeded it on new data (0.50%→1.72%; 0.43%→0.54%; §me-014).

Column "Rule" of Table 1 states which of these applied to each confirmation.

Descriptive p values and confirmation thresholds. A p value in this paper is confirmatory only when it is the committed primary judgement of a one-shot confirmation. Every other p (release-to-release comparisons, component comparisons on tuning and development sets) is descriptive: the sets had been seen, and many comparisons were made on them without a multiplicity correction; Holm is applied only within the family of tests that feeds one registered decision (§me-004). Even the confirmatory McNemar values are optimistic, because the test treats clauses as independent while clauses from one message or article are correlated. We did not re-analyse the historical confirmations with a clustered test, since that would be a post-hoc analysis; they are reported as they were judged.

"Not significantly worse" is not "non-inferior". The shipping standard below uses two different statements, and we keep them apart. *Not significantly worse* (exact McNemar $p \geq 0.05$, or net ≥ 0) is the absence of a detected loss; it puts no bound on the loss and is easily passed when discordant rows are few. *Meeting the non-inferiority margin* bounds the loss: it holds only when the 90% lower bound of Δtop-1 is above a margin fixed in advance. We use the first phrase only for the former and the second only for the latter.

Resolution. Before a confirmation, the minimum detectable effect is computed from the baseline's discordant-row count: with 61 discordant rows the 80%-power net gain at $\alpha = 0.05$ is +23 rows, and +27 at the Holm first step $\alpha = 0.01$. On small sets a paired exact sign test needs roughly 6:0 or 8:1 fixed:broken for $p < 0.05$, so a one-sentence difference on a 227-row or 150-row set is noise.

4.4 Exploration and confirmation

Since 2026-10-18 the rules distinguish two kinds of experiment. An *exploration* is a one-page registration (hypothesis, metric, sets, budget, stop rule) that touches only tuning- and development-layer sets, ships nothing and concludes only with a direction. A *confirmation* commits, before any number is read, a single primary judgement, its rule, the set, the budget, the minimum detectable effect and the diagnostics, and is run once on a never-seen judging set. Failing means "did not pass as registered"; no follow-up experiment may rescue the same confirmation, a variant goes back to exploration, and a repeat needs a new judging set. Changing a threshold, a filter, a set or a grid after seeing the result voids that confirmation. The rule "do not make contracts too strict" applies to product and UX rules only, never to statistical judging rules (§me-014). Where a rule

was changed after seeing a number we say so, and the affected change was re-confirmed on a new set where one remained; the one case where none remained, the shipped changes that had no one-shot confirmation, and the other deviations from these rules are listed in Appendix E.

4.5 The shipping standard

A new model, or any change that alters character selection, ships only if, against the current shipped build:

1. on the chat development set (discordtune, later CE3 `cdev`) it is not significantly worse (exact McNemar $p \geq 0.05$ or net ≥ 0) and broken/fixed ≤ 0.6;
2. the regression layer (typing76 in both profiles, dev302 in the written profile) is row-identical or loses ≤ 2 rows with net ≥ 0; dev302 in the chat profile is reported with every broken row categorised but does not block;
3. on cvtune-clean (later cvtune-clean-tw0) in the chat profile the 90% paired bootstrap lower bound of Δtop-1 is ≥ -1.0 percentage points;
4. on the user-reported set, both profiles combined, net > 0 and broken/fixed ≤ 0.6 (a lexicon-only pack may pass with 0 broken, including 0/0).

Condition 1 is a "not significantly worse" statement and condition 3 a non-inferiority margin in the sense of Section 4.3. Under the rules, an exploration that passes all four is confirmed once on a never-seen judging set before the product change is made; the shipped changes for which that step was not taken are listed in Appendix E. Condition 3 has a known weakness: cvtune-clean is a tuning-layer set on which the shipped (α, τ) were themselves selected, so the baseline has a home advantage there and the gate is lenient; we treat it as an exception to the rule that non-inferiority gates use the development layer, and do not gate on it any change whose values were selected on it (§me-017). The standard itself was changed several times, twice by relaxation after numbers had been seen; the changes that passed only under a relaxed rule were then confirmed on a fresh chat judging set (row 3 of Table 1). The dated history is in Appendix C. Where a shipped change failed one condition and shipped under a recorded exception, Appendix C says so and cites the record. The engineering controls (one CPU type per comparison, confirmation runs repeated on a second machine, hash checks of every input) are in Appendix E.

4.6 Provenance and reproducibility

With one exception, every number in this paper is copied from the project's public documents and none was recomputed for the paper: the research excerpt [49], which publishes, paragraph by paragraph, the parts of the development records that this paper cites; the cross-version benchmark [47]; the evaluation-set README [48]; the specifications [51]; and the release notes [50]. A reference of the form §rl-012 points to the anchor of that name in the excerpt. The excerpt is cited at commit `5a33e0ad`, which follows the v0.7.0 tag and adds four paragraphs of the plan (§plan-019 to §plan-022) without changing the existing ones; the other four documents are cited at the commit of the v0.7.0 tag. The v0.7.0 rows of Tables 2, 9 and 3, the v0.7.0 point of Figure 2 and the v0.7.0 latency come from the benchmark's result file for that release, the hold-out usage record and the release note; a v0.7.0 percentage is the published count divided by the size of the set. The exception is Table 4: its runs were made for this paper on the tagged commit (Section 5.3); a summary of the results and the script of the offline recomputation, `gate_offline.py` (`--off gate` for the margin-gate row, `--off soft` for the soft-gate row), are in the repository under `eval/ablation/v0.7.0/`

at commit `e88f4a36` [46]. The code, the lexicon and the model files are in the repository [35]; the public tuning and regression sets can be re-run with the evaluation CLI, and the model files are fetched by hash. Private evaluation sets (the author's own chat messages and a news set whose source is not licensed for redistribution) are described only by size and by the statistics that the documents publish; no sentence from them appears here. Where a statement in the source is marked as an inference rather than a measurement, we keep the marking.

5 Results

The section follows the three research questions. Section 5.1 gives the confirmatory evidence; Section 5.2 the release-level trends on development sets and the hold-out; Section 5.3 the ablation of the v0.7.0 system; Section 5.4 what moved accuracy and how reliable the measurements are; Section 5.5 two external comparisons. The history of each component, with project code names and dates, is in Appendix C.

5.1 One-shot confirmations

Table 1 lists every one-shot confirmation the project ran: one row per primary judgement, grouped by confirmation. It only collects existing records. Of the 13 confirmations, 8 held and 5 did not; counted by primary judgement, 12 of 17 passed. In four of the five failures the change had passed on tuning or development data before the run; in the fifth (row 5) the registered arm was not the arm that had been measured.

Notes on the table. In row 1 the main judgement passed and the broken-rate guard failed, so the cell was not exported; this is the case that turned the cap into an upper bound. In row 2 the 0.2% cell was admitted after a disclosed relaxation of the cap to 0.6% on seen data. Row 5 was run on a tuning-layer set that this line of work had not seen, and its registered arm was not the arm measured earlier (Appendix E). In rows 7–9 the sets are groups of teach, rare and common sentences written for the purpose. The arm of row 10 was confirmed but not shipped; row 11 is the arm that ships. Rows 12 and 13 confirm chat-profile cells on Wikipedia text, because the chat judging set had been used by row 6; their transfer to chat rests on shipping condition 1 alone. Where the record gives the absolute top-1 on the judging set, it is: row 4, 2,231→2,250; row 10, 2,262→2,260; row 11, 2,255→2,254; row 12, 2,119→2,164 (written) and 2,045→2,071 (chat); row 13, 2,100→2,149 (chat) and 2,208→2,222 (written); for the other rows the record gives paired counts only (§rl-039–§rl-042, §rl-053, §rl-076, §rl-077, §rl-087, §rl-088, §rl-090–§rl-094, §rl-108, §rl-110, §rl-113, §rl-114).

Judging sets that do not appear in the table: wikijudge3 is named only as retired together with the first two (§ip-005); the record does not say which decision used it. The first chat judging set appears as an old chat set and as the source of one diagnostic (§me-021, §ip-002); no one-shot confirmation on it is recorded. Before the regime, the re-ranker's fusion cell (α, τ, δ) for v0.4.1 was selected on tuning months and scored on a held-out judging month of 6,559 rows (Appendix C); that was not a committed registration and is not counted here.

5.2 Releases on development sets and on the hold-out

Table 2 is extracted from the repository's cross-version benchmark [47], which runs each release with its own shipped models and flags on frozen sets. Every set in it was seen while values were selected, and each release bundles several changes, so the table describes releases; it is not a test. The benchmark's descriptive p values are omitted; the fixed and broken counts of each release are in Table 9 (Appendix C). Figure 2 plots the two discordtune columns, and Table 3 gives the hold-out.

Table 1: The one-shot confirmations, in the order they were run. Rule I (improvement): net > 0, exact two-sided McNemar $p < 0.05$, ratio ≤ 0.65. Rule N (non-inferiority): lower bound (LB) of the 90% paired bootstrap interval of Δtop-1 ≥ -1.0 percentage point. F/B is fixed/broken against the arm named in Appendix C. †: run before 2026-10-18 under a written contract, without a committed registration. "Held" refers to the whole confirmation.

#	Change confirmed	Unseen set (rows), profile	Rule	F/B	p or bound	Held
1†	Written fusion cell, first re-selection	wikijudge (rows not recorded), written	main judgement plus a broken-rate cap of 0.5%	76/21	$p < 0.0001$; broken rate 1.72%	no
2†	Written fusion cell, second re-selection	wikijudge2 (2,818), written	main judgement plus broken-rate cap	39/11	$p = 0.0001$; 0.35%	yes
	conservative cell	same	same	20/5	$p = 0.004$; 0.14%	
3	Colloquial table behind a soft gate	chatjudge-2 (5,783), chat	I	108/39	$p = 1.1 \times 10^{-8}$; ratio 0.36	yes
4	Relaxed written fusion cell	wikijudge4 (2,820), written	I	67/48	$p = 0.0928$; ratio 0.72	no
5	One-word-skip class bridge	filtered chatclause (5,631), chat	net > 0, broken $\leq$ fixed/2	0/2	—	no
6	Reading term in the decoder	wikijudge5 (2,816), written	I	15/0	$p = 6.1 \times 10^{-5}$	yes
		cjudge-1 (5,639), chat	N	3/4	LB -0.089 pp	
7	Sentence-initial learning, version 1	project-written groups	common rows flipped ≤ 1	24 vs 7 rare rows right	3 flipped	no
8	version 2	second groups	same	21 vs 9	2 flipped	no
9	version 3	third groups	same	13 vs 7	1 flipped	yes
10	Re-ranker continuation training, first arm	wikijudge6 (2,844), written	N	9/11	LB -0.32 pp	yes
11	Re-ranker continuation training, shipped arm	wikijudge7 (2,857), written	N	5/6	LB -0.25 pp	yes
12	New fusion cells, standard strength	wikijudge8 (2,844), written	I	90/45	$p = 0.0001$; ratio 0.50	yes
		same set, chat profile	I	43/17	$p = 0.0011$; ratio 0.40	
13	New fusion cells, conservative strength	wikijudge9 (2,827), chat profile	N	68/19	LB $+1.20$ pp	yes
		same set, written	N	14/0	LB $+0.28$ pp	

Three things in these tables bear on the research questions. First, the largest step is v0.6.0, the release that rebuilt the bigram and the re-ranker with forum and chat-channel text: 336 fixed and 49 broken on the chat development set, and 33→70 of 80 on the frozen user-reported rows (Section 5.4). Second, that generation was not better on written text proper: on the Wikipedia tuning set the written profile went 2266→2243 (+78/−101) and the chat profile 2184→2160 (+80/−104), neither significant by the descriptive test, and the IT-pack check set fell: from 46 to 36 (chat) and from 39 to 27 (written) at the first stage of the rebuild, with the new bigram under the previous re-ranker, which are the figures the release note quotes, and by 3 fixed against 14 broken (chat) and 4 against 10 (written) in the final judging of the shipped combination (§rl-059, §rl-068; [50]). An earlier release, v0.4.4, regressed on the parliamentary set lytune (chat 4055→4010, written 4021→3979); the cause was an interaction between two decoder terms and was largely repaired two releases later (Section 5.4). Third, the development set and the hold-out differ widely in level: v0.6.0 is correct

Table 2: Cross-version benchmark: number of correct rows (top-1, lenient), chat profile unless noted. "user-rep." is the 80-row frozen user-reported set; the last column is the written profile on the same discordtune clauses. Releases run from v0.4.0 (2026-10-10) to v0.7.0 (2026-11-11). All sets were seen during value selection.

Version	dev302 (302)	typing76 (76)	user-rep. (80)	cvtune (3,791)	discordtune (4,958)	same, written
v0.4.0	241	66	17	3360	4209	4143
v0.4.1	259	69	25	3441	4299	4172
v0.4.2	259	69	25	3441	4299	4172
v0.4.3	259	69	25	3445	4314	4191
v0.4.4	259	70	24	3446	4303	4177
v0.4.5	259	70	24	3446	4303	4177
v0.4.6	262	72	28	3448	4304	4178
v0.4.7	262	72	28	3448	4304	4218
v0.5.0	269	72	33	3485	4358	4267
v0.6.0	280	74	70	3530	4645	4588
v0.6.1	280	74	70	3530	4645	4588
v0.7.0	282	74	70	3536	4664	4646

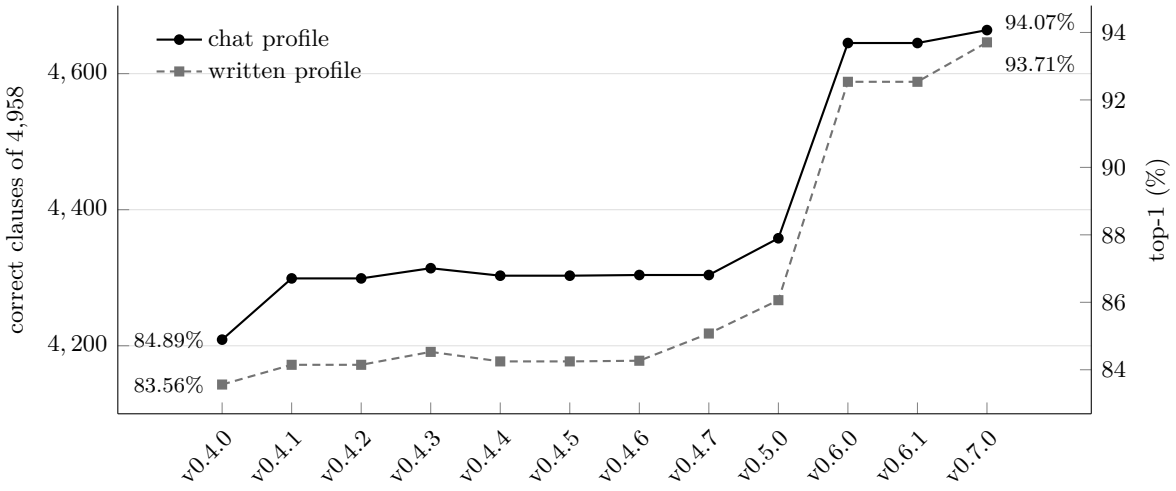


Figure 2: First-choice accuracy by release on discordtune, the 4,958 clauses of the author's chat, in the chat profile and in the written profile on the same clauses. Every point is a cell of the two discordtune columns of Table 2; the right axis re-expresses the same counts as a percentage of 4,958, and the four labelled percentages are the end points quoted in the abstract and the conclusion. discordtune is a development-layer set that was seen while values were selected (Table 8), so the curves are descriptive; each release bundles several changes.

on 4,645 of 4,958 clauses of the author's chat and on 124 of 150 clauses of the public colloquial half. The two sets differ in source and register as well as in exposure to value selection, and no experiment separates the two effects, so we neither read the difference as a measured amount of overfitting nor rule overfitting out. One recorded data point speaks against a pure selection effect: an earlier pipeline scored 86.71% on discordtune and 88.61% on a judging month that had not been used for selection (Section 5.5).

The last rows of the tables are v0.7.0, the release this paper describes. Against v0.6.1 the chat

Table 3: Hold-out, run by an independent agent before each tag (release-monitoring evidence; see Section 4.2). Up to v0.4.6 the 227-row original set; from v0.4.6 the 300-row refreshed shard (news and talk); from v0.6.0 the two 150-row halves (public talk, chat profile; private news, written profile). The second v0.5.0 row is the same release measured on the two halves, on a later commit, as the baseline for v0.6.0. "Use" is the use number of the talk half and news half in the public record; v0.7.0 was measured twice, before and after the rerank-protect list was added, with the same result, and the talk half was then replaced (Section 4.2). Numbers across shard changes are not comparable. Fixed/broken and the exact McNemar p are against the previous tagged release; releases that changed nothing (v0.4.2, v0.4.5) are omitted.

Version	Set	Chat	Written	Use (talk / news)
v0.4.0	227	180 (79.30%)	184 (81.06%)	not recorded
v0.4.1	227	191 (+11/−0, $p = 0.000977$)	187 (+3/−0, 0.25)	not recorded
v0.4.3	227	191 (0/0)	189 (+2/−0, 0.5)	not recorded
v0.4.4	227	191 (+3/−3)	186 (+2/−5, 0.453)	not recorded
v0.4.6	227 / 300	191 (+3/−3) / 234	187 (+6/−5) / 230	not recorded
v0.4.7	300	234 (0/0)	233 (+7/−4, 0.549)	1
v0.5.0	300	237 (+7/−4, 0.549)	241 (+8/−0, 0.0078)	4
v0.5.0	150+150	115 (76.67%)	125 (83.33%)	with v0.6.0
v0.6.0	150+150	124 (+10/−1, 0.0117)	128 (+3/−0, 0.25)	5 / 1
v0.6.1	150+150	124 (0/0)	128 (0/0)	6 / 2
v0.7.0	150+150	127 (+3/−0, 0.25)	126 (+1/−3, 0.625)	7, 8 / 3, 4

development set gained 31 and lost 12 rows in the chat profile and gained 100 and lost 42 in the written profile. The written profile also rose on the Wikipedia tuning set (2243→2303, +97/−37; chat profile 2160→2186, +46/−20), on dev302 (271→280, +10/−1) and on the frozen user-reported rows (61→69 of 80, +9/−1; the chat profile stayed at 70). Two public sets moved the other way: the parliamentary set lytune in the chat profile (4430→4401, +33/−62; written 4384→4403, +114/−95) and the Wikipedia typo set in the written profile (137→131 of 155, +1/−7). On the hold-out the colloquial half went from 124 to 127 and the news half from 128 to 126; neither difference is significant ($p = 0.25$ and 0.63; the exact test needs about 6:0 or 8:1 fixed to broken, Section 4.3). The descriptive p values of the other sets are in Appendix C [47].

5.3 Ablation of the v0.7.0 system

The component results of Section 5.4 were each measured when the component was introduced, against the model generation of that time, so they do not add up and do not say what each part contributes to the present system. Table 4 answers that with leave-one-out runs on the commit of the v0.7.0 tag at the standard strength, each row removing one part with an existing switch of the evaluation CLI. Three notes are needed to read it. Without the colloquial term the fusion uses the model file's own chat triple $(0.3, 2, 0.25)$ instead of the chat cell $(1.1, 2, 0)$ (the chat trigram table stays on), so that row is the joint effect of the term and its cell. The word-class and Kneser–Ney rows are removed from the decoder-only system, because the CLI refuses to remove them while the re-ranker is loaded: its cells were selected with them on. The two gate rows have no switch; they are recomputed offline from the per-candidate scores of the full-system run by the shipped rule of Equations (4) and (6), which first reproduced the run's choice on every row, with $\tau = \infty$ and $\delta = 0$ for the margin gate and $g_i = 1$ for the soft gate and everything else unchanged. The runs are descriptive and are the only numbers computed for this paper.

Table 4: Leave-one-out ablation of the v0.7.0 system (commit `56979b2e`, the v0.7.0 tag): correct rows (top-1, lenient), with fixed/broken in parentheses against the system named at the head of the block. Standard strength, the shipped flags (left context and the three default packs), one run on one machine. cvtune-clean-tw0 and wikitune-clean-tw0 are public; discordtune is the filtered private set, for which only aggregate counts are reported and no score file is published. The written profile has no colloquial term (n/a); the unigram baseline, a separate code path without packs, writes no per-row results and so has no fixed/broken. All three sets were seen while values were selected: the table is descriptive.

System	cvtune-clean-tw0 (chat, 2,992)	wikitune-clean-tw0 (written, 2,182)	discordtune (chat, 4,919)
Full system	2,771	1,677	4,633
without the re-ranker and its colloquial term	2,635 (21/157)	1,553 (43/167)	4,516 (39/156)
without the colloquial term	2,697 (19/93)	n/a	4,582 (34/85)
without the margin gate ($\tau = \infty$, $\delta = 0$)	2,771 (0/0)	1,675 (0/2)	4,632 (0/1)
without the soft gate ($g_i = 1$)	2,775 (11/7)	n/a	4,628 (5/10)
without the trigram table	2,768 (3/6)	1,617 (14/74)	4,628 (3/8)
without the reading term	2,763 (0/8)	1,655 (0/22)	4,616 (0/17)
Decoder only (the second row)	2,635	1,553	4,516
without the word-class back-off	2,632 (4/7)	1,544 (4/13)	4,513 (7/10)
without the Kneser–Ney continuation value	2,606 (15/44)	1,510 (17/60)	4,499 (21/38)
Unigram baseline (lexicon only, no packs)	2,186	953	3,884

Four things can be read from the table; every p below is a descriptive exact McNemar value. First, of the parts removed one at a time, the re-ranker with its colloquial term is the largest on all three sets (the n-gram model itself is not one of those parts; the gap between the decoder-only system and the unigram baseline of the last row, which also lacks the packs, is several times larger): without it the system loses a net 136, 124 and 117 rows ($p < 0.0001$ each), and removing only the colloquial term, with its cell, loses 74 and 51 rows on the two chat sets ($p < 0.0001$). Second, the decoder's parts differ by register. The trigram table is worth a net 60 rows in the written profile ($p < 0.0001$) and is not detectable in chat (-3, $p = 0.51$; -5, $p = 0.23$). In the decoder-only system the Kneser–Ney value is worth 29, 43 and 17 rows ($p = 0.0002$, < 0.0001, 0.036) and the word classes 3, 9 and 3 ($p = 0.55$, 0.049, 0.63). Third, removing the reading term fixes no row and breaks 8, 22 and 17 ($p = 0.0078$, < 0.0001, < 0.0001); these sets have generated readings, on which the term's cost on secondary readings cannot appear (Section 8). Fourth, at the standard strength the margin gate is almost never the binding constraint on any of the three sets: it holds the decoder's first choice on 802 of the 2,992 public chat rows, 509 of the 2,182 written rows and 2,012 of the 4,919 private chat rows, and without it the fusion would change none of the first, two of the second and one of the third, all three for the worse ($p = 0.5$ and $p = 1$). The row measures the threshold τ only, because the standard cells already have $\delta = 0$; the conservative strength's δ was not ablated. Fixing the soft gate of the colloquial term at 1 takes the public chat set from 2,771 to 2,775 correct rows (11 fixed, 7 broken, $p = 0.48$) and the private one from 4,633 to 4,628 (5 fixed, 10 broken, $p = 0.30$); the chat cell was selected with the gate on, so the row is descriptive only and does not say what a system tuned without the gate would do. Neither row varies the fusion weight α, the replacement margin δ or the place where the colloquial table enters, and it is those that the results under RQ2 in Section 5.4 vary.

5.4 What moved accuracy, and how reliable the measurements are

Table 5 summarises the component experiments. Each row compares a change with the build shipped at the time, on the set that decided it; the rows span five model generations, so the effects are historical and cannot be summed. The last column points to the confirmation of Table 1, where one exists.

Table 5: Component experiments, each against the build shipped at the time (historical; not additive). "Set" gives the set and its role. p values are descriptive exact McNemar values unless the row has a confirmation. "Conf." is the row of Table 1.

Component	Set (role)	Effect	p or bound	Conf.
Word-class back-off	cvtune (tuning)	net +21 chat, +26 written	0.002	—
Kneser–Ney continuation value	four tuning cells, 13,236 rows	net +33; wikitune written 38/20	0.025 (that cell)	—
Character-LM re-ranker, 14.4 M parameters	discordtune (chat)	+67 rows, 95% CI [29, 105]	0.0008	—
Fusion cell re-selected on the author's register	judging month, 6,559 rows	125/16 (earlier values 82/7)	—	—
Trigram table, written profile	wikitune (tuning)	108/61	0.0004	—
Class weight lowered to 0.2	lytune (development)	100/73 chat; 130/65 written	0.048; < 0.0001	—
Written fusion cell re-selected	wikijudge2 (judging)	39/11	0.0001	2
Colloquial table behind a soft gate	discordtune; chatjudge-2	74/43; 108/39	0.0053; 1.1×10^{-8}	3
Bigram rebuilt with forum and chat-channel text	discordtune (development)	4,366→4,534, 201/33	—	—
Re-ranker retrained on the same text	discordtune (development)	4,558→4,620, 88/26	—	—
Reading term in the decoder	wikijudge5; cjudge-1	15/0; 3/4	6.1×10^{-5}; LB -0.089 pp	6
Re-ranker continuation training and new fusion cells	discordtune (development), standard strength	29/11	0.0064	11–13
Sentence-initial learning	third group of test sentences	13 vs 7 rare rows right	—	9

RQ1: in-register text against a larger neural model. The step that dominates Table 2 is the model generation that added the text of a public chat channel and of a public Taiwanese forum to the counts of the bigram and to the training text of the re-ranker. The rebuilt bigram alone fixed 201 and broke 33 rows of the chat development set and took the user-reported set from 58 to 95 correct rows (chat) and from 51 to 99 (written); the re-ranker retrained on the same sources, compared by U_2 over three seeds on a tuning set, won in every seed where a re-run of the old recipe lost in every seed, and added 88 fixed against 26 broken on top of the new decoder. These comparisons are on development, regression and tuning sets, which had been seen, and this generation has no one-shot confirmation (Appendix E). Selecting the best-scoring half of the new text by cross-entropy difference was worse than taking all of it. The price was paid on written text: the same re-ranker lost on the Wikipedia tuning set (1,634→1,616, 25/43, $p = 0.039$), which we read as the cost of more colloquial training text (not tested), and six rows of the written regression set broke and shipped as a recorded exception. By contrast, no attempt to enlarge the neural part passed (Section 6). The small re-ranker itself was worth having: against the decoder alone it gained 67 rows on the chat development set, far less than on public sets (dev302 chat 241→277), which we attribute to the decoder being already 84.9% correct on that set (not tested separately). Decoder

structure gave smaller steps: word classes and the Kneser–Ney value were each worth a few tens of rows on tuning sets, their interaction caused the one significant regression of a release, and lowering the class weight recovered the written loss entirely and about six tenths of the chat loss.

RQ2: the decision rule. Three results say that how the fusion is set matters as much as the model behind it. (i) Values chosen on public written sets twice failed to transfer to chat; re-selecting the triple on the author's own register gave 125 fixed against 16 broken on a judging month where the earlier values gave 82 against 7. (ii) The colloquial table as a separate decoder term fixed 168 and broke 89 rows at the decoder level, but once the full pipeline was re-scored it broke 92 rows of the chat development set; moved into the fusion, anchored on the decoder's first choice and passed through a soft gate, it gave 74/43 there and 108/39 on the unseen chat judging set. A control arm that only strengthened the fusion without the table gave 61/38 on the judging set and 44/52 on the development set, so we attribute the gain to the table rather than to the stronger fusion (inferred from the control arm; not separately tested). (iii) For the last re-ranker, a full scan found that under the old written cell the shipped and the new weights are almost identical (3,356 against 3,349 of 4,182 rows) and that both gain about the same from a stronger cell (3,420 against 3,424): the written gain is almost entirely the fusion cell. The new cells were then confirmed on two unseen sets (rows 12 and 13 of Table 1).

RQ3: retraining churn and selection optimism. When the re-ranker was retrained from scratch without a faulty conversion table, it lost to the shipped weights in all three seeds. A same-seed paired comparison showed that the loss is no larger than retraining churn: any two re-rankers swap about 90–100 rows of the chat tuning set and 65–80 rows of the chat development set, with a net within ± 12 ($p > 0.2$). Continuation training from the shipped weights cut the swap to 21–33 rows; further recipes and seeds did not bring it below the seed-noise floor (the continued model against its own re-seeds swaps 39 and 33 rows). A separate check found that the shipped fusion cell had been selected for one seed: the same recipe re-seeded scores U_2 of -50 and -63 against it on the selection set. A model change of a few tens of rows is therefore not distinguishable from a re-seed unless seeds are replicated and the cell is re-selected per arm. Selection optimism shows in the same records: the broken-to-fixed ratio of a selected cell was 0.33 on its selection set and 0.58 on the development set, 0.45 and 0.60 for the next, 0.42 and 0.72 for the relaxed written cell on its judging set; a broken rate of 0.50% on tuning data became 1.72% on new data. Table 1 is the aggregate: 5 of 13 confirmations failed, four of them after passing on seen data.

A conversion table as a root cause. Two user-reported rows were wrong because the text used to train the re-ranker had been converted by a phrase table of a Chinese-conversion tool (`TWPhrases` of OpenCC [9]), which rewrites 127 lexicon words, 71 of them frequent; on 94 constructed sentences the shipped re-ranker broke 7 of 49 decoder-correct rows (chat) and 7 of 46 (written), where the model continued on unconverted text broke 1 and 0. The same table had also converted the references of the tuning sets, which were rebuilt without it. This is a data defect, not a modelling result, but it changed more user-visible rows than several of the model experiments.

5.5 External comparisons

The 0c model as a re-ranker on Shanjie's candidate pool. This comparison answers a narrow question: if the MIT-licensed model `0c-sequence-d512` takes the place of Shanjie's re-ranker in a fusion over the same candidate pool (the decoder's eight best paths), does the fusion choose better? It is not a comparison of the two input methods. The inputs differ: 0c sees the last

64 characters of left context and the readings; Shanjie's re-ranker sees at most two characters of left context and no readings. In the first run, on the v0.4.1 decoder, each model's fusion values were selected by the same method. On the author's chat the 0c fusion was better than the re-ranker then shipped (discordtune 89.27% against 86.71%, +189/−62; judging month 90.78% against 88.61%, +200/−58). On the public sets, in the chat profile, the result depended on the set: the 0c fusion was worse on the three large ones (wikitune 73.65% against 76.76%, +46/−134; lytune +191/−247; cvtune +54/−62, not significant), level on dev302 (259 correct rows each) and better on the small ones (the first 84 user-reported rows 36 against 25, +16/−5, $p = 0.027$; typing76 72 against 69; function words 161 against 157; wikitypo 136 against 134). Its file is 130.7 MB in fp32 against about 14.6 MB in int8 (§ip-002; [47]). In the second run, on the v0.6.0 decoder, the 0c fusion (cell selected on the chat tuning set at a 0.4% upper bound) was compared with the shipped pipeline, which besides its re-ranker has the colloquial table and a reading term in the fusion. On the 4,919 rows of the filtered discordtune the 0c fusion was $+49/-13$ against the decoder and $+39/-110$ against the shipped pipeline (net −71, ratio 2.82), and the shipped pipeline $+140/-33$ against the decoder. 0c could not read 34 of the rows; the record gives the count but not how those rows were scored in the fused arm. Only the fusion of 0c with the decoder was measured; 0c on top of the shipped re-ranking was not (§rl-071, §rl-072). Within this candidate pool and these fusion settings, the advantage 0c had on the author's chat at v0.4.1 was gone at v0.6.0.

A rule-based converter. Ruhi, a rule-based open-source converter, was run on the three public sets with its own readings and no context. Whole-sentence correct: user-reported 51/168 against Shanjie 131 (chat with context) and 127 (without); dev302 154/302 against 280/279; typing76 39/74 against 72/72. Rows only Ruhi got right: 14, 3 and 1; four of the 14 were rows with 長 read ㄓㄤˇ, later fixed by the decoder reading term (§rl-069).

6 Negative results

Table 6 classifies the experiments that did not pass by the kind of failure; the complete list, with code names and every recorded number, is in Appendix D. The four kinds support different conclusions. A *regression* is evidence against the idea under the conditions tested. *Insufficient evidence* means that the effect was smaller than the experiment could detect, which is not evidence of no effect. *Resource-infeasible* says nothing about accuracy at a larger budget. *Procedure invalid* means that the result cannot be read as registered. A failed registration is not by itself a closed line of work; closing a line is a separate recorded decision (§rl-052).

Table 6: Negative results by kind of failure. Numbers are those of the record (Appendix D); "rows" are net rows on the author's chat development set unless another set is named.

Experiment	Result against its criterion
Regression: the effect went the wrong way	
On-device foundation model as judge	chat development set 83.5%→75.4%; candidate order changed 49–55% of its choices
Larger re-ranker (30.2 M parameters)	−30 rows; −97 against the 14.4 M model
Listwise training objective	net −38 against the shipped re-ranker
Distilled student with 63 characters of context	−39
Colloquial table as a decoder term	128/92 in the full pipeline: too many broken under the standard
Counting bigrams per reading	−1,457 rows on lytune chat; reading annotation wrong in 64 of 157 sampled positions

Experiment	Result against its criterion
Selecting new text by cross-entropy difference	-35 written on a slang check set ($p < 0.0001$)
Insufficient evidence: below what the experiment could detect	
Re-ranker retrained on added colloquial text (first attempt)	estimated net $+16$ on the chat judging set against a minimum detectable effect of $+32$
Larger re-rankers on the trigram decoder	registered Holm test could not be passed at the effect sizes seen
64-character context, retrained or continued	U_2 0, -9, -32 against seed noise -6, -17: no direction
Re-ranker retrained from scratch on unconverted text	U_2 -52, -44, -46, within same-recipe churn
Relaxed written fusion cell (confirmation)	net $+19$, $p = 0.0928$, ratio 0.72
Students distilled from the 0c model	-2, -2, $+7$ for three variants (two others -35, -41)
Resource-infeasible: latency, memory or size	
Local LLM re-ranking	p95 848 ms against a 300 ms limit (and 91.4% against a 95% gate)
Cloud LLM judges	best 92.7% at p95 3,146 ms; best under one second 92.4%
Largest re-ranker	did not fit an 8 GB GPU
Unpruned trigram table	1,133.3 MB, the only size that passed at its selected values
Procedure invalid: the result cannot be read as registered	
A "one broken ≈ ten fixed" selection weight	best achievable net $+17$ to $+23$ of 4,091 rows, the size of platform-and-seed noise
One-word-skip class bridge (confirmation)	the registered arm was not the arm measured earlier; 0/2
Word-sense classifier for 在/再	mixed logarithm bases and a distorted class prior found in review; stop rule triggered
Web-forum and public-domain text as evaluation text	5.3–12.7% of sampled rows unusable; 13.8% still needing correction after filtering; abandoned

What the negative results say. (i) Within the conditions tested (a 14-to-30-million parameter character model, two characters of context in the shipped format, the training text listed in Section 8), making the neural part larger, giving it longer context, training it discriminatively or distilling it from a stronger teacher did not pass. Several of these are "insufficient evidence" and not "regression": they could not be told apart from retraining churn, and one was infeasible on the available GPU. (ii) Selection weights and metrics need a ceiling-and-noise check before use. Under the "one broken ≈ ten fixed" weight the 14.4 M model scored 70 fixed and 10 broken, a weighted utility $U_{10} = \text{fixed} - 10 \times \text{broken} = -30$, and semantic correction shipped off by default; computed afterwards, the best net attainable under that weight even when choosing values on the judging set was $+17$ to $+23$ of 4,091 rows, the same size as the difference between two otherwise identical runs (32 rows). (iii) Judges that pick among candidates are capped by candidate coverage and by latency before model quality matters: the top 16 candidates contained the answer 94.7% of the time at the start of the project, and the local and cloud judges that were timed ran at a p95 of 510 to 3,146 ms against a 300 ms limit. (iv) Rules for reading annotation, word sense and data selection lost to the plain counts they were meant to refine.

7 Conclusion

Shanjie's selection pipeline is conventional: an n-gram decoder and a 14.6-million-parameter re-ranker behind a margin gate. The evidence of about six weeks says where its accuracy came from.

On the author's chat development set, first-choice accuracy rose from 84.89% to 94.07% between v0.4.0 and v0.7.0 (83.56% to 93.71% in the written profile on the same clauses) and the frozen user-reported set from 17 to 70 correct of 80; the largest step is the release whose bigram and re-ranker were rebuilt with forum and chat-channel text. On written text proper (wikitune) the written profile went from 75.52% to 81.46% over the same releases; the v0.6.0 model generation lost 23 rows there (2266→2243, +78/−101, not significant) and v0.7.0 gained 60 (2243→2303, +97/−37). All of these are development-set numbers. The confirmatory evidence is the 13 one-shot confirmations of Table 1, of which 8 held; the v0.6.0 model generation is not among the changes they cover (Appendix E). On the release-monitoring hold-out, v0.5.0→v0.6.0 moved the colloquial half from 115 to 124 ($p = 0.012$, the fifth use of that half) and the newly activated news half from 125 to 128 ($p = 0.25$), and v0.6.1→v0.7.0 moved them to 127 and 126 ($p = 0.25$, 0.63). Attempts to make the neural component larger, longer-context, discriminatively trained or distilled from a stronger teacher failed their registered criteria or could not be told apart from retraining churn, within the conditions tested. In the leave-one-out ablation of the v0.7.0 system, which is descriptive, removing the re-ranker with its colloquial term costs a net 117 to 136 rows on each of three sets, the trigram table matters in the written profile only, and the margin threshold at the standard strength changes at most two rows (Table 4). In the stripped replay of the key path the decoder's per-key p95 was 2.06 ms for v0.5.0 and 2.23 ms for v0.6.0 (the latter not in an idle window), and 1.91 ms for v0.6.1 and 1.93 ms for v0.7.0 in one session on another machine at a starting load average of 1.9; the re-ranker runs off the key path, and full-application latency was not measured. The rules, the confirmation ledger and the deviations from the rules are published so that others can check them; the limits of the evidence are in Section 8. Next steps (§rl-115, §rl-052, §ip-003, §ip-006): rebuild the bigram and the re-ranker together once the forum crawl is complete and the conversion repair table is in place; estimate $P(\text{reading} \mid \text{character})$ properly for the reading term; make use of the 64-character context the shell already returns; and ship in-app counters so that "correct offline" can be measured in the app.

8 Limitations

One user. The chat sets that decided shipping up to v0.6.1 (discordtune, chatclause, chatjudge-k) are the author's own messages: only clauses the author wrote enter them. The CE3 sets read the public channels of one designated server, the author's own Threads and Facebook posts, and small samples of Forum A replies and of two public microblogs; cjudge-1 is 2,880 of the author's clauses plus 2,759 Forum A clauses (§plan-014, §me-021, §rl-087; Section 9). The method is explicit about this ("chat first, written only as a gate") and the sets are large (about 5,000 clauses each), but nothing here shows transfer to other users' registers beyond the Forum A half of cjudge-1.

Tuning sets overlap training text. 507 cvtune rows (13.4%) and 671 wikitune rows (23.7%) occur verbatim in the character LM's training text. Every value selected on these two sets (the frozen α, τ of the first re-ranker and later cells) may therefore be biased upward; the decisions that shipped them were taken on discordtune, which is not in the training text (§rl-018, §plan-014).

The user-reported set is one-sided and has served as a clue. It is collected only when a sentence came out wrong, and its words have been used as hints for lexicon additions; the project's rules forbid scoring rows that served as hints, but the 17→70 series on the frozen 80 rows is evidence that reported errors were repaired, not of generalisation to unseen typing (§me-012).

Readings are generated. Every tuning, development and judging set has readings produced by a tool that gives single characters their primary reading. A term that penalises secondary readings therefore cannot be charged on these sets; in the only set with real keystrokes (user-reported, 327 single characters) four characters carried a penalised reading, and one of them broke a row (§rl-055, §me-019). Keystroke-level effects (typing speed, candidate window, learning store) are measured only by the true-engine replay tool, not by the offline CLI; the two agreed row for row in every check so far.

Offline versus in-app. 29% of user-reported typos were correct offline when first measured. Splitting by the version reported against, the chat "correct offline" share was 41% for reports up to v0.2.1, 27% for v0.3.0–v0.4.5 and 11% for v0.4.6–v0.4.7 (inference: much of the bucket is errors later versions fixed, not an offline/in-app gap; no re-run on old versions). The cross-version benchmark does not simulate learning, the asynchronous re-rank timing or the shell's context reading, so in-app gains from semantic correction may be smaller than tabulated (recorded as an inference). In-app counters and a "report this sentence" feature are planned but not shipped (§ip-002, §ip-003, §me-017).

Sets seen during tuning. All sets except each one-shot judging set have been seen when selecting values; the benchmark page states that only the hold-out was not. The hold-out is a release-monitoring set whose summary statistics have been reported repeatedly (Section 4.2); it has 150 rows per half; between the three shards of the refreshed set the same version differs by up to 13 sentences (about 4%), so hold-out numbers are not compared across shards [47, 48].

dev302. 299 of its 302 rows were written by the project or by an LLM, and it reversed the sign of real-chat results twice; it blocks only the written profile and is otherwise reported (§me-017).

Statistical regime. Several thresholds were changed after seeing data; each case is dated in Appendices C and E. Scores of the small model differ across CPU types at the 10^{-6} level, so all arms of a comparison run on one CPU and, since 2026-10-29, confirmation and shipping-standard numbers are taken on two machines (§me-015).

Data. Sources are described by type and size only. The bigram is counted on Chinese Wikipedia (the first 200,000 articles, weight 1), a colloquial pool (Common Voice zh-TW and Tatoeba sentences and synthetic colloquial sentences, weight 5), the public g0v `#rand0m` channel (weight 5), and posts and replies from a public Taiwanese forum (Forum A; replies weight 5, post bodies weight 1) [50]. The re-ranker is trained on the same sources plus OpenSubtitles v2018 Traditional-Chinese subtitles (4,432,866 sentences, 42,814,742 Han characters after cleaning; Section 3.5). The released model files contain counts and weights, not sentences; the release note of every shipped model states that no private conversation entered it [50]. Forum A's identity and its raw text are not disclosed, and no evaluation set derived from it is published (Section 9).

Scope. macOS only, Traditional Chinese (Taiwan) only, Zhuyin only; a Windows core compiled and tested on 2026-11-06, and the built-in Microsoft Zhuyin IME was measured as a reference (175 key-state cells, 173 pixel-identical across two runs), but no Windows product exists (§rl-116, §rl-119).

Version. The system described is v0.7.0. The cross-version benchmark measured it on a commit that differs from the tagged one in none of the paths the evaluation specification binds to a release (the build files, the core, the CLI, the data, the variant table, the shell and the scripts) [47, 51]. Its release-level numbers are development-set and monitoring numbers like those of every earlier release. Of the changes since v0.6.1, the re-ranker weights, the fusion cells, the decoder reading term and the sentence-initial learning rule each have a one-shot confirmation (Table 1); the sentence-initial estimate of the colloquial term, the rule for tied readings, the rerank-protect list and the third default pack have none (Appendix E). The ablation of Table 4 was run once, on one machine, without the second-machine repeat used for confirmations; for the private set only aggregate counts are published, so its column, the two gate rows included, cannot be recomputed from public files.

No same-day comparison with other input methods. The only comparison with existing IMEs is the day-one snapshot of Appendix F, taken on a set that later served as a regression set, with one IME carrying a learning state. The paper therefore makes no claim that Shanjie selects better than any other input method.

9 Ethics statement

Private chat evaluation sets. The chat sets that selected values and decided shipping up to v0.6.1 (discordtune, chatclause, chatjudge-k) are built from the author's own messages: text the author wrote in Discord server channels, on Threads and on Facebook, with direct messages excluded and any message containing personal data or keys dropped whole (§plan-014). The CE3 sets (from 2026-10-27) read only the public channels of one designated Discord server, plus the author's own Threads and Facebook posts and small samples of Forum A replies and of two public microblogs; CE3 cjudge-1 is 2,880 clauses of the author's text and 2,759 clauses of Forum A replies, each part proofread on a 10% sample, with rows hit by nickname or form-of-address keywords removed (§me-021, §rl-087). The judging sets are drawn from a fixed month window (2015-07 to 2026-07), with 2026-09 permanently excluded and text typed with Shanjie from 2026-10 on never used as a reference (§ip-003). The sets are not published; the paper reports only their sizes and the statistics computed on them, and the project's evaluation tools print only numbers for private sets [51].

Shipped models and private text. The shipped models are counted or trained on public text only; no private message enters any released file, and the release note of each model states so [50]. The evaluation specification records the rule behind this: a group chat contains other people's utterances, so such text may be used for evaluation and tuning but never to train a model that ships with the installer [51].

Forum text. Posts and replies from a public Taiwanese forum (Forum A) are used for word counts, for training, and for the Forum A part of the unpublished CE3 judging set (Table 8); this paper refers to the forum only by that code name, its raw text is not distributed, and no evaluation set derived from it is published ([50], §rl-087; Section 8).

News text. The private news development set and the news half of the hold-out come from a news source not licensed for redistribution; the files are kept outside the repository, which records only their row counts and SHA-256 hashes, and they are never used for training [48].

Hold-out handling. The hold-out sets are run only by independent agents (LLM sessions that did not implement the change), only before a release tag, and only their summary statistics are returned; every executor brief forbids reading the hold-out files [48].

References

[1] 0c-biggo. 0c-ime (agpl-3.0; model weights mit), 2026. URL https://github.com/0c-biggo/0c-ime. Accessed 2026-11-10.

[2] amzxyz. rime-wanxiang, 2026. URL https://github.com/amzxyz/rime-wanxiang. Accessed 2026-11-11.

[3] azooKey. Zenzai: neural kana-kanji conversion in AzooKeyKanaKanjiConverter, 2024. URL https://github.com/azooKey/AzooKeyKanaKanjiConverter/blob/main/Docs/zenzai.md.

[4] azooKey. AJIMEE-Bench, 2026. URL https://github.com/azooKey/AJIMEE-Bench. Accessed 2026-11-11.

[5] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. An empirical investigation of statistical significance in NLP. In *Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning*, pages 995–1005, 2012. URL https://aclanthology.org/D12-1091/.

[6] Avrim Blum and Moritz Hardt. The Ladder: A reliable leaderboard for machine learning competitions. In *International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1006–1014, 2015. URL https://proceedings.mlr.press/v37/blum15.html.

[7] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Naz Sepah, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Dmitriy Serdyuk, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. *arXiv preprint arXiv:2103.03098*, 2021. URL https://arxiv.org/abs/2103.03098.

[8] Peter F. Brown, Vincent J. Della Pietra, Peter V. deSouza, Jennifer C. Lai, and Robert L. Mercer. Class-based n-gram models of natural language. *Computational Linguistics*, 18(4): 467–479, 1992.

[9] BYVoid. OpenCC: Open chinese convert, 2026. URL https://github.com/BYVoid/OpenCC. Accessed 2026-11-11.

[10] Stanley F. Chen and Joshua Goodman. An empirical study of smoothing techniques for language modeling. *Computer Speech & Language*, 13(4):359–393, 1999. doi: 10.1006/csla.1999.0128.

[11] Zheng Chen and Kai-Fu Lee. A new statistical approach to Chinese Pinyin input. In *Proceedings of the 38th Annual Meeting of the Association for Computational Linguistics*, pages 241–247, 2000. URL https://aclanthology.org/P00-1031/.

[12] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[13] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, 2018. URL https://aclanthology.org/P18-1128/.

[14] Cynthia Dwork, Vitaly Feldman, Moritz Hardt, Toniann Pitassi, Omer Reingold, and Aaron Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248): 636–638, 2015. doi: 10.1126/science.aaa9375.

[15] Funmula / BigGo. 0c zhuyin input method: character-selection technology page and the MIT-licensed 0c-sequence-d512 model, 2026. URL https://0c.biggo.com/technology/. Accessed 2026-11-10.

[16] Jianfeng Gao, Joshua Goodman, Mingjing Li, and Kai-Fu Lee. Toward a unified approach to statistical language modeling for Chinese. *ACM Transactions on Asian Language Information Processing*, 1(1):3–33, 2002. doi: 10.1145/595576.595578.

[17] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems 30*, 2017. URL https://arxiv.org/abs/1705.08500.

[18] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 1321–1330, 2017. URL https://arxiv.org/abs/1706.04599.

[19] Lars Hellsten, Brian Roark, Prasoon Goyal, Cyril Allauzen, Françoise Beaufays, Tom Ouyang, Michael Riley, and David Rybach. Transliterated mobile keyboard input via weighted finite-state transducers. In *Proceedings of the 13th International Conference on Finite State Methods and Natural Language Processing (FSMNLP 2017)*, pages 10–19, 2017. URL https://aclanthology.org/W17-4002/.

[20] Yafang Huang, Zuchao Li, Zhuosheng Zhang, and Hai Zhao. Moon IME: Neural-based Chinese Pinyin aided input method with customizable association. In *Proceedings of ACL 2018, System Demonstrations*, pages 140–145, 2018. URL https://aclanthology.org/P18-4024/.

[21] Zhongye Jia and Hai Zhao. KySS 1.0: a framework for automatic evaluation of Chinese input method engines. In *Proceedings of the Sixth International Joint Conference on Natural Language Processing*, pages 1195–1201, 2013. URL https://aclanthology.org/I13-1170/.

[22] Zhongye Jia and Hai Zhao. A joint graph model for Pinyin-to-Chinese conversion with typo correction. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1512–1523, 2014. URL https://aclanthology.org/P14-1142/.

[23] Mike Tian-Jian Jiang, Cheng-Wei Lee, Chad Liu, Yung-Chun Chang, and Wen-Lian Hsu. Robustness analysis of adaptive Chinese input methods. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 53–61, 2011. URL https://aclanthology.org/W11-3509/.

[24] Reinhard Kneser and Hermann Ney. Improved backing-off for M-gram language modeling. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, pages 181–184, 1995.

[25] Philipp Koehn. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 388–395, 2004.

[26] Taku Kudo, Toshiyuki Hanaoka, Jun Mukai, Yusuke Tabata, and Hiroyuki Komatsu. Efficient dictionary and language model compression for input method editors. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 19–25, 2011. URL https://aclanthology.org/W11-3503/.

[27] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[28] levelel. librime-witogram, 2026. URL https://github.com/levelel/librime-witogram. Accessed 2026-11-11.

[29] Pierre Lison and Jörg Tiedemann. OpenSubtitles2016: Extracting large parallel corpora from movie and TV subtitles. In *Proceedings of the 10th International Conference on Language Resources and Evaluation (LREC 2016)*, pages 923–929, 2016.

[30] Sven Martin, Jörg Liermann, and Hermann Ney. Algorithms for bigram and trigram word clustering. *Speech Communication*, 24(1):19–37, 1998. doi: 10.1016/S0167-6393(97)00062-9.

[31] metasequoiaime. pinyin-ime-reranker-4m (model card), 2026. URL https://huggingface.co/metasequoiaime/pinyin-ime-reranker-4M. Accessed 2026-11-11.

[32] Mahdi Milani Fard, Quentin Cormier, Kevin Canini, and Maya Gupta. Launch and iterate: Reducing prediction churn. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/dc5c768b5dc76a084531934b34601977-Paper.pdf.

[33] Robert C. Moore and William Lewis. Intelligent selection of language model training data. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics (Short Papers)*, pages 220–224, 2010.

[34] Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *Proceedings of the 37th International Conference on Machine Learning*, pages 7076–7087, 2020. URL https://arxiv.org/abs/2006.01862.

[35] Nanako0129. 善解 shanjie: an open-source macos zhuyin input method, 2026. URL https://github.com/Nanako0129/shanjie.

[36] Hermann Ney, Ute Essen, and Reinhard Kneser. On structuring probabilistic dependences in stochastic language modelling. *Computer Speech & Language*, 8(1):1–38, 1994. doi: 10.1006/csla.1994.1001.

[37] OpenVanilla. McBopomofo: Zhuyin input method for macos (MIT; phrase data from libtabe), 2026. URL https://github.com/openvanilla/McBopomofo. Accessed 2026-11-11.

[38] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In *International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5389–5400, 2019. URL https://proceedings.mlr.press/v97/recht19a.html.

[39] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348, 2017. URL https://aclanthology.org/D17-1035/.

[40] RIME. librime: Rime input method engine, the core library, 2026. URL https://github.com/rime/librime. Accessed 2026-11-10.

[41] Hisami Suzuki and Jianfeng Gao. A comparative study on language model adaptation techniques using new evaluation metrics. In *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language Processing*, pages 265–272, 2005. URL https://aclanthology.org/H05-1034/.

[42] Minghuan Tan, Yong Dai, Duyu Tang, Zhangyin Feng, Guoping Huang, Jing Jiang, Jiwei Li, and Shuming Shi. Exploring and adapting Chinese GPT to Pinyin input method. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1899–1909, 2022. URL https://aclanthology.org/2022.acl-long.133/.

[43] Yee Whye Teh. A hierarchical Bayesian language model based on Pitman-Yor processes. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 985–992, 2006.

[44] The Chewing Project. libchewing: the intelligent phonetic input method library, 2026. URL https://codeberg.org/chewing/libchewing. Accessed 2026-11-10.

[45] Hiroyuki Tokunaga, Daisuke Okanohara, and Shinsuke Mori. Discriminative method for Japanese kana-kanji input method. In *Proceedings of the Workshop on Advances in Text Input Methods (WTIM 2011)*, pages 10–18, 2011. URL https://aclanthology.org/W11-3502/.

[46] Nanako Tsai. Shanjie v0.7.0 leave-one-out ablation: results and the offline recomputation of the two gate rows (`eval/ablation/v0.7.0/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/e88f4a364d414780678999ee471096b40bb6212c/eval/ablation/v0.7.0. As of commit e88f4a36; the directory was added after the v0.7.0 tag.

[47] Nanako Tsai. Shanjie cross-version benchmark: result files (`eval/bench/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/eval/bench. As of commit 56979b2e (the v0.7.0 tag). The public sets are shown at https://shanjie.nyanako.com/benchmark.

[48] Nanako Tsai. Shanjie evaluation sets: roles, hashes and hold-out usage record (`eval/README.md`), 2026. URL https://github.com/Nanako0129/shanjie/blob/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/eval/README.md.

[49] Nanako Tsai. Shanjie research excerpt (english): the development-record paragraphs cited by the paper, 2026. URL https://github.com/Nanako0129/shanjie/blob/5a33e0adbd09bb4885c53db132e978103030adf1/docs/research-excerpt.en.md. As of commit 5a33e0ad (2026-11-11; it follows the v0.7.0 tag and adds four paragraphs of the plan). In-text references of the form §rl-012 are anchors in this file.

[50] Nanako Tsai. Shanjie release notes for the input method and the model files (`docs/releases/`), 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/docs/releases.

[51] Nanako Tsai. Shanjie specifications (`docs/spec/`): decoder, models, semantic re-rank, learning, lexicon and packs, evaluation, privacy, 2026. URL https://github.com/Nanako0129/shanjie/tree/56979b2e2f6c7fd7a7288c38ed772d26f297e0e4/docs/spec.

[52] Emiel van Miltenburg, Chris van der Lee, and Emiel Krahmer. Preregistering NLP research. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 613–623, 2021. URL https://aclanthology.org/2021.naacl-main.51/.

[53] vChewing. vChewing (唯音) input method, 2026. URL https://github.com/vChewing/vChewing-macOS. Accessed 2026-11-11.

[54] Jinghui Xiao, Qun Liu, Xin Jiang, Yuanfeng Xiong, Haiteng Wu, and Zhe Zhang. PERT: A new solution to Pinyin to character conversion task. *arXiv preprint arXiv:2205.11737*, 2022. URL https://arxiv.org/abs/2205.11737.

[55] Yuqing Xie, Yi-An Lai, Yuanjun Xiong, Yi Zhang, and Stefano Soatto. Regression bugs are in your model! measuring, reducing and analyzing regressions in NLP model updates. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6589–6602, 2021. URL https://aclanthology.org/2021.acl-long.515/.

[56] Sijie Yan, Yuanjun Xiong, Kaustav Kundu, Shuo Yang, Siqi Deng, Meng Wang, Wei Xia, and Stefano Soatto. Positive-congruent training: Towards regression-free model updates. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14299–14308, 2021.

[57] Hui Zhang and David Chiang. Kneser-Ney smoothing on expected counts. In *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics*, pages 765–774, 2014. URL https://aclanthology.org/P14-1072/.

[58] Zhuosheng Zhang, Yafang Huang, and Hai Zhao. Open vocabulary learning for neural Chinese Pinyin IME. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 1584–1594, 2019. URL https://aclanthology.org/P19-1154/.

A Lexicon, notation and learning rules

This appendix gives the parts of the pipeline that Section 3 only summarises.

A.1 Lexicon

The base lexicon is McBopomofo's. On top of it sit an overlay built from Chinese Wikipedia and Wiktionary titles (CC BY-SA 4.0; a stricter filter, "V2t", kept 342,761 words after dropping simplified-only characters, because taking all 1.04 million titles made selection worse by polluting candidates with proper names; §rl-003); tone-sandhi reading variants for 一/不/法 generated from Ministry of Education rules (1,302 rows, later 1,502 with a neutral-tone tolerance layer); a names layer (Taiwanese streamers, channels, shops, companies, non-Taiwan Chinese-language celebrities); internet slang collected in two rounds (507 and 910 words); and three optional packs that are on by default, an anime/game pack (44,552 words from Wikidata), an IT+AI pack (1,016 words from

Table 7: Symbols. Counts are weighted expected counts over word lattices, rounded.

Symbol	Value	Meaning
r, r_j		typed syllables; a reading
w, v, u		lexicon entries: the word, its predecessor, the one before
π, $S(\pi)$, $E(\pi)$		a path, its score, its closing (sentence-end) term
λ_p	0.5 / 0.7	model-vs-prior weight, chat / written
$lp(w)$, $lp'(w)$	$\log_{10}$	lexicon prior; the prior after learning
$s(r, w)$	$\log_{10}$	raw file score of an entry before capping
$c(\cdot)$, $t(\cdot)$, N		bigram or trigram count; pre-pruning context total; unigram total
D, D_3	0.75	absolute discounts (bigram, trigram)
$pb(w)$, $pb_{\mathrm{KN}}(w)$		unigram back-off value; its Kneser–Ney replacement
θ, $N_\theta(w)$, β_p	1, —, 1 / 0.75	continuation threshold and count; KN weight, chat / written
$C(w)$, K, μ, $\varepsilon_{\mathrm{cls}}$	512, 0.2, 0.1	word class, number of classes, class weight, class smoothing
$e(w)$, $q(w \mid v)$		class emission; class-smoothed back-off
T_p, F	0.5	profile's trigram table; trigram floor
$\eta(w)$, η_{dem}, η_{sound}, η_{adj}	2.5, 2.0	demotion and tolerance penalties
$\rho(w)$, $\xi(r, w)$, λ_r, $c_{\max}$	2.0, 4	reading term, its log-ratio, weight, cap
y_i, n_c, ng_i	$n_c \leq 8$	re-rank candidates, their number, their decoder scores
k, ll_i, $\tilde{ll}_i$	$\lvert k \rvert \leq 2$	context key; character-LM score; its colloquially fused value
(α, τ, δ)	per strength	fusion cell: fusion weight, decoder-margin threshold, replacement margin
f_i, U		fused score of candidate i; usable candidates
P_{CL}, lc_i, col_i, o_i, κ	0.5	colloquial table and its path score, penalised score, pack-word count, per-word penalty
μ_{CL}, T_g, g_i, γ	0.5, 1.0, —, 0.25	colloquial weight, gate temperature, gate, per-pack-word weight
ω, ε_ℓ	6.0	learning record weight (decayed); learning bonus
b_i, ℓ_i, $\varepsilon_{\mathrm{fine/mid/coarse}}$	4, 4, 2	sentence-initial record share, learning term, its layer weights
U_2		fixed $-2\times$broken, the re-ranker selection score

NAER terminology, Wikipedia conversion groups and a hand list) and, from v0.7.0, a beauty and skincare pack (539 words) [50]. Overlay words are scored by a capped prior, never above their corpus frequency (Equation (7)). Pack names the pack builder marks as forced bypass the general statistics so that they are not torn apart by common characters (Section A.3).

A.2 Notation

Table 7 lists every symbol used in more than one subsection; each means one thing throughout. A lexicon entry is a (reading, string) pair; we write w for the entry and let the reading be implicit, so $lp(w)$, $\eta(w)$ and $\rho(w)$ are functions of the entry (the same string under another reading is another entry), and we write (r, w) only where the reading itself is operated on.

A.3 Lexicon priors, packs, demotion and tolerance

Every lexicon entry (r, w) has a file score $s(r, w)$ in $\log_{10}$: McBopomofo's for base rows, a per-length constant for overlay rows. Let $c(w)$ be the model's unigram count of w and N the weighted unigram total. Whether a word is "in the overlay" is decided by its string: every entry of a word listed in the overlay file is capped, a base-lexicon row of the same string included. The prior the decoder uses is

$$lp(w) = \begin{cases} s(r,w) & \text{not an overlay word; a forced or model-unseen pack row;} \\ \min\big(s(r,w), \log_{10}(c(w)/N)\big) & \text{other overlay word, } c(w) > 0; \\ s(r,w) - 1.0 & \text{other overlay word, } c(w) = 0, \end{cases} \tag{7}$$

so an overlay word can never score above its corpus frequency. A pack row the pack builder marks *forced* (a name that must beat the common-character spelling of the same reading) keeps $s(r, w)$ whatever $c(w)$ is and is scored as unknown to the model: $P(w \mid v) = \text{back}(v)\, 10^{lp(w)}$, no class term, and the next word is scored as if its context were unknown. A demotion table lists $(r, w, \eta_{\text{dem}})$ rows for readings the dictionary does not attest (and sensitive words); their η_{dem} is part of $\eta(w)$ in Equation (1). Tolerant readings add to $\eta(w)$: $\eta_{\text{sound}} = 2.5$ when one syllable of the entry's reading was reached through a fuzzy pair (ㄣ / ㄥ, ㄓ / ㄗ, ㄔ / ㄘ, ㄕ / ㄙ; off by default) and $\eta_{\text{adj}} = 2.0$ when a syllable the lexicon lacks was read through an adjacent key.

A.4 Kneser–Ney continuation side file

The side file carries, for every vocabulary word w, the continuation count $N_\theta(w)$: the number of distinct left contexts v whose rounded count $c(v, w) \geq \theta$, with $\theta = 1$. Variant spellings of one word form one class, counted once both in N_θ and in the denominator below. The back-off value becomes

$$pb_{\text{KN}}(w) = \beta_p \frac{N'(w)}{\sum_{w''} N'(w'')} + (1 - \beta_p)\, 10^{lp(w)}, \qquad N'(w) = N_\theta(w) + 1, \tag{8}$$

and replaces $pb(w)$ in every branch of Equation (2) for vocabulary words; a word outside the vocabulary keeps 10^{lp}, and $\langle/s\rangle$, which has no lexicon prior, keeps its end-of-sentence rate eos/N as back-off value with or without the side file. β_{chat} is the file's value (1) and $\beta_{\text{written}} = 0.75$. The $+1$ keeps $pb_{\text{KN}} > 0$ and makes the value monotone in N_θ for a fixed lp. Because the counts are rounded expected counts, this is the threshold-and-integer approximation of continuation counting, not the fractional-count variant of [57], and the discount D is not re-estimated (§rl-017).

A.5 Word-class back-off

The class file gives each of the 40,000 most frequent words a class $C(w)$ from a Brown-style exchange clustering with $K = 512$ classes (the table has $K + 3$ slots: $\langle s\rangle$ and $\langle/s\rangle$ carry their own classes), a class bigram table $P_C(d \mid c)$ smoothed with $\varepsilon_{\text{cls}} = 0.1$, and an emission $e(w) = \text{uni}(w)/\sum_{w' \in C(w)} \text{uni}(w')$, where $\text{uni}(w)$ is the word's unigram count in the clustering corpus (not the model's $c(w)$). Where the bigram has no kept entry and both words have a class,

$$q(w \mid v) = (1 - \mu)\, pb(w) + \mu\, P_C\big(C(w) \mid C(v)\big)\, e(w), \qquad \mu = 0.2, \tag{9}$$

with pb already the Kneser–Ney value when the side file is on, and $q(w \mid v) = pb(w)$ when either word has no class. The class term therefore never touches a kept bigram; it redistributes back-off mass. The weight was $\mu = 0.8$ at introduction and 0.2 since v0.4.6 (Appendix C; §rl-014, §rl-030). Words that keep their count across a rebuild keep their class; only new words are assigned.

A.6 Trigram table

The trigram tables (written, 7,700,964 entries; chat, 950,000 entries) store, for stored contexts (u, v), the rounded counts $c_3(u, v, w)$ of kept successors and the pre-pruning total $t_3(u, v)$, with discount $D_3 = 0.75$. With $P_2(w \mid v)$ the bigram of Equation (2) (with the profile's β_p),

$$\begin{aligned} P_3(w \mid u, v) &= \max\Big(\frac{\max(c_3(u, v, w) - D_3, 0)}{t_3(u, v)} + \mathrm{back}_3(u, v)\, P_2(w \mid v),\ F\, P_2(w \mid v)\Big), \\ \mathrm{back}_3(u, v) &= 1 - \sum_{w' :\, c_3(u,v,w') > 0} \frac{c_3(u, v, w') - D_3}{t_3(u, v)}, \end{aligned} \tag{10}$$

with the floor $F = 0.5$; when (u, v) is not stored, or u is unknown, $P_3 = P_2$. At the start of a sentence the history is $(\langle s\rangle, \langle s\rangle)$, so the first word does use the table; u is unknown when the stretch follows a fixed word, because the engine passes only that one word of left context, so the first word after a fixed word is scored by the bigram, and a one-word stretch after a fixed word also closes with the bigram. The word term of Equation (1) uses P_3 whenever the profile has a table; the closing term uses $P_3(\langle/s\rangle \mid u, v)$ with pb the end-of-sentence rate. The table is memory-mapped read-only, so its pages are clean file-backed pages outside the process footprint; the floor F is a constant of the decoder, not of the file (§rl-032, §rl-037).

A.7 Reading term in the decoder

Evaluation readings and the lexicon's placeholder scores had been letting rare readings of common single characters win: "變快" typed after "否則" came out as "便會" (§rl-081). For a single-character entry w used under syllable r, let $s(r, w)$ be the raw file score and $s^*(w) = \max_{r'} s(r', w)$ its best reading. Define, taking the first case that applies,

$$\begin{aligned} \xi(r, w) &= \begin{cases} 0 & (r, w) \in \mathcal{X}, \\ \log_{10} M(r, w) - \max_{r'} \log_{10} M(r', w) & s(r, w) = -6.8 \text{ (the floor)}, \\ 0 & (r, w) \in \mathcal{S}, \\ s(r, w) - s^*(w) & \text{otherwise}, \end{cases} \\ \rho(w) &= \lambda_r \max\big(\xi(r, w), -c_{\max}\big), \end{aligned} \tag{11}$$

with $\lambda_r = 2.0$, $c_{\max} = 4$; $\mathcal{S} = \{(ㄧˊ, 一), (ㄧˋ, 一), (ㄅㄨˊ, 不)\}$ the sandhi readings; $\mathcal{X}$ the twelve pairs whose dictionary-first reading the lexicon lists second (朴、嗎、率、哩、還、和、癌、賃、波、相、嚇、教); and $M(r, w) = 10^{s(r,w)} + \sum 10^{s(\mathrm{word})}$ over every multi-character base word containing w read r at that position (the "mc" estimate for floor pairs; a sandhi pair that sat at the floor would take the mc value, a case the shipped lexicon does not contain). $\rho(w) = 0$ for multi-character entries and for a word the learner lifted (the term can cost up to $\lambda_r c_{\max} = 8$, a learned lift gives at most 6, and a reading the user taught must stand). The term is in the path score, so the beam prunes on it, in both profiles, at all three strengths and with semantic correction off; it replaced an earlier reading term that lived inside the re-rank fusion, which is the form measured in v0.6.1 (Appendix C).

v0.7.0 adds one case, applied after the exemptions. For a character that has at least two single-character readings, all with $\xi = 0$, that differ in more than tone, let $m(r)$ be the sum of $10^{s(\mathrm{word})}$ over the base lexicon's multi-character words containing the character read r; a reading with $\log_{10}\big((m(r) + 10^{-9})/(\max_{r'} m(r') + 10^{-9})\big) \le -1$ takes that value as ξ, and the others keep 0. In the shipped lexicon this separates 132 (reading, character) pairs, for example the two readings

of 莎, which the lexicon scores equal; on the public development set, the typing test and the user-reported set in both profiles it changed one row, for the better [50, 51].

A.8 Learning from re-selection

A record is a (context key, reading, word) triple with a weight ω that halves every 14 days, $\omega_{\text{today}} = \omega \cdot 0.5^{\max(\text{today}-\text{day},\,0)/14}$ (a clock that went back never grows it); records below 0.5 are inactive and records below 0.05 are dropped. Below, ω is the decayed weight. The context key is the last at most two Han characters before the word (a sentinel at sentence start, where the key also carries the next syllable); a single character is learned only under an exactly matching two-character key or a sentence-start key. For an entry with an active record under the current key,

$$lp'(w) = \max\Big(lp(w),\ lp^*(r) + \varepsilon_\ell \,\frac{\omega}{\omega+1}\Big), \qquad \varepsilon_\ell = 6.0, \tag{12}$$

where $lp^*(r)$ is the best capped prior under the reading r; the global layer (a word learned under two contexts spreading to a third) has its own bonus $\varepsilon_{\text{global}} = 0$ and is therefore off since v0.3.0 (Table 10). A lifted entry is scored with $pb = 10^{lp'}$ and *without* the Kneser–Ney substitution or the trigram table (both would shrink the lift back to its $(1-\lambda)lp'$ share), and it takes no reading term. A re-pick halves the displaced word's record, so a heavier record outranks a lighter one. A 2–4-character string spelled out by successive single-character re-selections is learned as a new word with prior $lp_{\text{new}} = -3 + 2 \cdot \frac{\omega}{\omega+1}$, scored as unknown to the model.

Sentence-initial single characters (the SINIT slice) are not lifted in the decode that builds the candidate pool; they enter the re-rank fusion instead. Because a taught character may not be among the pool's n_c paths at all, the engine runs a second decode in which the taught character is lifted to the best prior of its reading, takes from it the best path that starts with that character, restores that path's unlifted score and priors, and puts it into the pool's last slot (never the first) if the pool does not already contain its text. For a stretch whose first word is a single character, let $b_i = \frac{\omega}{\omega+1}$ of the record of candidate i's first character under the longest matching key layer (fine: the sentinel plus the next three syllables, the third replaced by an end marker when the composition ends there; mid: two; coarse: one). The learning term of Equation (4) is

$$\ell_i = \begin{cases} \varepsilon_{\text{fine}}\,(b_i - b_0) & \text{fine layer, } \varepsilon_{\text{fine}} = 4, \\ \varepsilon_{\text{mid}}\,\max(b_i - b_0, 0)\,\mathbb{1}[ll_i > ll_0] & \text{mid layer, } \varepsilon_{\text{mid}} = 4, \\ \varepsilon_{\text{coarse}}\,\max(b_i - b_0, 0)\,\mathbb{1}[ll_i > ll_0] & \text{coarse layer, } \varepsilon_{\text{coarse}} = 2, \end{cases} \tag{13}$$

with $\mathbb{1}[\cdot]$ the indicator, the layers never adding up. The fine layer works in both directions (a learned anchor is held against a better-scoring unlearned candidate); the mid and coarse layers only lift, and only where the character LM already leans the same way. When the decoder is sure ($ng_0 - ng_1 \geq \tau$), only candidates with $\ell_i > \ell_0$ may replace the first choice, and by δ like any other. This rule passed its pre-registered confirmation on 2026-11-03 after two failed versions (Section C.4; §rl-094).

B Evaluation sets

Table 8 lists the evaluation sets with their sizes, the origin of their readings and their roles (Section 4.2).

Table 8: Evaluation sets. Private sets are described by size only. "Readings" states whether readings were typed by a person or generated from the reference by the project's tool.

Set	Register	Rows	Readings	Role / source
dev302	both	302	confirmed	regression (written only since 2026-10-20); 299 rows project- or LLM-written
typing76	both	76	confirmed	regression; project-written typing test, CC0
probe (s2r)	both	83	generated	regression; sandhi reading variants
fw (function words)	both	186	generated, checked	regression; 在/再, 的/得/地, 做/作
user-reported	both	187 (80 frozen for the benchmark)	as typed	regression; real typos, CC0; one-sided (collected only when wrong)
wikitypo	both	155	generated	regression
cvtune / cvtune-clean / -tw0	chat	3,791 / 3,286 / 2,992	generated	tuning; Common Voice zh-TW + Tatoeba 20%
wikitune / -clean / -tw0	written	2,827 / 2,181 / 2,182	generated	tuning; Wikipedia articles 300,000–304,999
lytune	written	4,811	generated	development, warning only; Legislative Yuan speeches
discordtune (`cdev`)	chat	4,958 (4,919 filtered)	generated	development; the author's chat, private
chatclause (`ctune`)	chat	5,761 (5,631 filtered)	generated	tuning; private
chatjudge-2	chat	5,783	generated	judging, used once; private
CE3 cjudge-1	chat	5,639 (2,880 own + 2,759 Forum A)	generated	judging, used once; private
wikijudge-k	written	2,816–2,857 each	generated	judging, one decision each
newstune	written	2,000	generated	development; private news (72 Wikinews + 1,928)
hold-out talk (shard-1)	chat	150	generated, 40 checked	release monitoring (8 uses up to v0.7.0, then replaced by the next shard); public colloquial text
hold-out news-1	written	150	generated	release monitoring (4 uses up to v0.7.0); private (30 Wikinews + 120)

C Component history

This appendix keeps the record of each component in the order it was built, with the project's code names and dates, and with the descriptive p values as recorded. Rows 1–13 of Table 1 correspond to the experiments named here rerank-formal (rows 1, 2), P4d (3), DECIDE-1 (4), BRIDGE-2 (5), R1g (6), SINIT revisions 7, 8 and 9 (7–9), SR-TW-Q (10), SR-TW-R (11), FUSE-1 (12) and FUSE-2 (13). Model families are named by function with a code, a version and a size (Appendix H).

C.1 Baselines and the first weeks

With McBopomofo's base lexicon and a unigram decoder, the 302-row development set is 51.7% correct (52.6% lenient; oracle@32 96.7%) and the original 227-row hold-out 41.4% (44.5%; 95.6%) [48]. Adding the Wikipedia/Wiktionary overlay raised overall oracle@64 from 96.9% to 97.8%. The first bigram (Wikipedia 50,000 articles, $\lambda = 0.5$, colloquial corpus $\times 5$) took the development set from 163 to 231 correct and the private chat error rate from 23.3% to 18.9%. In the paired-test regime that followed, interpolating with a unified prior and adding the sentence-end term fixed a net +112 rows on cvtune and +207 on wikitune ($p = 10^{-16}$, 10^{-31}); growing Wikipedia from 50,000 to 200,000 articles added +29 and +71 ($p = 0.004$, 3×10^{-8}); the overlay cap fixed 11 and broke 0 on wikitune ($p = 0.001$). None of these tuning-set gains transferred to the private chat validation set at first look (fixed 32, broken 32), which is what led to collecting a private chat tuning set of 4,958 clauses: on it the best round-4 setting was 4,066 against 4,045 for the old best (fixed 207, broken 186, $p = 0.31$). Of the 892 wrong clauses, 865 were ranking errors and 27 had no correct candidate in the top 64; the commonest confusions were 再→在 (31 clauses), 吃→喫 (29), 做→作 (20), 錢→前 (18), 在→再 (17), 甚→什 (16) (§rl-005, §rl-007, §rl-008, §rl-009, §rl-010, §rl-012).

The bigram shipped in v0.1.0 (2026-10-04); the hold-out at that point was chat 175/227 (77.1%), written 181/227 (79.7%), oracle@64 226/227 (99.6%), meeting the coverage target; the model was 80,040,411 bytes, loaded in 238 MB RSS with 1.18 ms per-key p95. Sandhi reading variants added +36 ($p = 5.6 \times 10^{-9}$) on the cvtune probe and +62 ($p = 6 \times 10^{-17}$) on the discordtune probe without regressing original readings. Removing simplified-character residue from the lexicon and corpus (3,489 overlay rows; seven marker characters 125,635→0 occurrences) gave cvtune chat +15 ($p = 0.02$) (§plan-008, §plan-010).

C.2 History of the shipping standard

The standard of Section 4 is itself a result. Condition 1 was "not significantly worse and broken $\leq$ the shipped value" on 2026-10-19; dev302 chat was demoted to report-only and cvtune-clean non-inferiority added on 2026-10-20 after an experiment lost 7 rows on dev302 chat (the bound was set at -1.0 pp rather than -0.3 pp because a normal approximation put the rejection rate of a truly neutral change at about 80% for -0.3 pp versus about 20% for -1.0 pp); the broken count became "broken ≤ 26 or ratio ≤ 0.5" on 2026-10-20 after seeing P4b; the ratio was relaxed to ≤ 0.6 the same evening after seeing P4d (0.58); and the absolute "broken ≤ 26" alternative was dropped on 2026-10-27 when the chat set changed (§rl-047, §rl-049, §rl-053, §me-021). The two relaxations made after seeing numbers are recorded as such, and the changes that passed only under the relaxed rule were confirmed on chatjudge-2.

C.3 Cross-version details

In the written profile discordtune went 4143 (v0.4.0) → 4172 (v0.4.1, +30/−1) → 4191 (v0.4.3) → 4177 (v0.4.4) → 4178 (v0.4.6) → 4218 (v0.4.7, +138/−98, $p = 0.011$) → 4267 (v0.5.0, +69/−20)

→ 4588 (v0.6.0, +396/−75, $p = 9.6 \times 10^{-54}$) → 4646 (v0.7.0, +100/−42, $p = 1.2 \times 10^{-6}$); in the chat profile v0.7.0 was +31/−12 ($p = 0.0054$). On the Wikipedia tuning set the v0.6.0 model generation was not better: wikitune chat 2184→2160 (+80/−104, $p = 0.0897$) and written 2266→2243 (+78/−101, $p = 0.0998$), both not significant. The IT-pack check set fell from 46 to 36 (chat) and 39 to 27 (written) at stage 1 of the rebuild (the new bigram under the previous re-ranker; the release note quotes these figures), and the final judging of the shipped combination recorded 3/14 (chat) and 4/10 (written) on it, negative at all three strengths (§rl-059, §rl-068; [50]). v0.7.0 rose on the Wikipedia tuning set, chat 2160→2186 (+46/−20, $p = 0.0019$) and written 2243→2303 (+97/−37, $p = 2.2 \times 10^{-7}$), and on dev302 written (271→280, +10/−1, $p = 0.012$) and the frozen user-reported rows written (61→69, +9/−1, $p = 0.021$); it fell on lytune in the chat profile (4430→4401, +33/−62, $p = 0.0038$; written 4384→4403, +114/−95, $p = 0.21$) and on the Wikipedia typo set written (137→131, +1/−7, $p = 0.070$). The v0.4.4 release regressed significantly on the parliamentary set lytune (chat 4055→4010, +40/−85, $p = 7.0 \times 10^{-5}$; written 4021→3979, +54/−96, $p = 0.00076$); the attribution and repair are in Section C.4.

Table 9: Fixed/broken against the previous release, for the releases that changed a set (same sets and profiles as Table 2; a dash means the set was row-identical or its paired counts are not in the sources used here).

Version	dev302	typing76	user-rep.	cvtune	discordtune (chat)	discordtune (written)
v0.4.1	18/0	3/0	9/1	84/3	114/24	30/1
v0.4.3	1/1	—	—	6/2	20/5	28/9
v0.4.4	3/3	1/0	2/3	19/18	40/51	72/86
v0.4.6	6/3	2/0	5/1	18/16	50/49	86/85
v0.4.7	—	—	—	—	—	138/98
v0.5.0	9/2	—	7/2	53/16	88/34	69/20
v0.6.0	14/3	2/0	38/1	95/50	336/49	396/75
v0.7.0	4/2	—	—	21/15	31/12	100/42

C.4 Components

Word classes (S2k). On model-v3 the class term gave cvtune-native +21 (chat) and +26 (written), $p = 0.002$, with the other sets changing non-significantly and the hold-out +2/0; the per-key p95 in the shipped engine was 3.597 ms (load below core count). The changed rows were mostly "homophones whose neighbours never co-occurred" (期中/其中, 戦機/戦績), and the broken rows clustered on 再/在 and homophonous verbs (接上/街上); we read this as consistent with a bigram-plus-class model seeing only adjacent words (an inference, not tested; §rl-014, §rl-015).

Kneser–Ney side file. The selected $\theta = 1, \beta = 1$ fixed a net +33 rows over the four tuning cells (13,236 rows, 0.25%); the only significant cell was wikitune written 2,131→2,149 (+38/−20, $p = 0.025$); the hold-out moved chat 180→182 (+6/−4, $p = 0.75$) and written 184→183 (+5/−6, $p = 1.0$), and none of five fixed probe sentences was fixed. It shipped bundled with model-v6 (§rl-017).

The small character LM (S7a, S7c). Two sizes were trained on the same 255,373,634 weighted tokens (Wikipedia 240,750,484 characters, colloquial 2,924,630 characters ×5): S (5.66 M parameters, final loss 4.190 nats/token) and M (14.4 M, 3.993). With frozen (α, τ) selected on the public

tuning sets, M passed the registered gate on discordtune chat: +67 rows out of 4,958 (+1.35 pp, threshold +1.0), $p = 0.0008$, 95% CI [29, 105], ΔCER −0.23 pp; S did not (+31, $p = 0.149$). The gains on public sets were far larger (dev302 chat 241→277, user-reported 15→31) than on the author's chat; we attribute this to the decoder being already 84.9% correct there, leaving harder errors (not tested separately). The int8 export (14.6 MB) reproduced the float32 result in Rust (+66, $p = 0.0010$), and the hold-out chat went 180→200/227 ($p = 0.0003$). The engine integration added 0.16 ms to the per-key p95 (about 2.6 ms) and reduced the fraction of already shown characters rewritten by later keys from 14.5% (decoder alone) to 9.3%, with 2.2% of keys showing a flash-back (§rl-018, §rl-019, §plan-014).

Computing the values on the author's register (E2, v0.4.1). Because tuning on public written sets had twice produced values that did not transfer, a chat tuning/judging set was built from the author's own messages, with the judging month fixed to 2026-08 (6,559 rows). The previously shipped conservative values (chosen under a "one broken ≈ ten fixed" weight) scored fixed 82, broken 7, net +75, broken rate 0.11%; the re-selected values $(\alpha, \tau, \delta) = (0.3, 2, 0.25)$ scored 125/16, net +109, 0.24% (upper bound 0.37%), and shipped as v0.4.1 with semantic correction on by default. On the benchmark v0.4.1 fixed 114 and broke 24 on discordtune ($p = 3.2 \times 10^{-15}$) and took the user-reported set from 17 to 25 of 80 (§plan-014).

Trigram in the written profile (v0.4.7). The pruned table (7,700,964 entries, 4,101,808 contexts) re-mapped to model-v6 with Kneser–Ney off gave, on the Python reference, tuning +64 (205/141, $p = 0.0007$). With the shipped re-ranker on top, the written profile fixed 108 and broke 61 on wikitune ($p = 0.0004$), 164/123 on discordtune ($p = 0.018$), 132/97 on lytune ($p = 0.024$), with no set significantly worse. The table costs 136 MB of RSS (423 vs 287 MB) and 0.01 ms of p95 (2.15 vs 2.13 ms). The v0.4.7 benchmark shows written wikitune 2183→2232 (+101/−52, $p = 9.2 \times 10^{-5}$), cvtune 3416→3440 (+75/−51, $p = 0.04$), lytune 4044→4080 (+117/−81, $p = 0.0127$) and discordtune 4178→4218 (+138/−98, $p = 0.011$); chat was row-identical (§rl-031, §rl-033, §rl-035).

Class weight and the lytune regression (v0.4.6). A factorial attribution on lytune split the v0.4.3→v0.4.4 loss (−45 chat, −42 written) into stages. The model rebuild itself changed nothing (3/3 and 7/7 swaps); the class term alone and Kneser–Ney alone were small and non-significant; their interaction was the regression: with both on, the class term cost −39 (88/127, $p = 0.009$) and −56 (103/159, $p = 0.0007$). A sweep of $\mu \in \{0, 0.2, 0.4, 0.6\}$ in the shipped configuration found $\mu = 0.2$ best by the registered rule: lytune +27 (100/73, $p = 0.048$) chat and +65 (130/65, $p < 0.0001$) written, with discordtune and the user-reported set unchanged within noise. v0.4.6 shipped $\mu = 0.2$ and recovered the written loss entirely and about six tenths of the chat loss (§rl-028, §rl-029, §rl-030).

Written re-rank strength (rerank-formal, SJ-SR-6-14M). Re-selecting the written (α, τ, δ) on the trigram decoder, the standard cell chosen at a 0.5% broken-rate cap on tuning data passed its primary judgement on a never-seen Wikipedia set wikijudge ($+76/-21$, $p < 0.0001$) but failed the broken-rate gate on the union of other sets (229/13,292 = 1.72%), with lytune at 2.81%; it was not exported. A second round selected cells at caps from 0.5% to 0.1%, admitted the 0.2% cell (0.7, 2, 1) to confirmation after a disclosed relaxation to 0.6% on seen data, and confirmed both it and the conservative cell on a fresh wikijudge2 (2,818 rows): $+39/-11$, net +28, $p = 0.0001$, broken rate 0.35%; and $+20/-5$, $p = 0.004$, 0.14%. The new written standard $(0.7, 2, 1)$ became

SJ-SR-6-14M; on the hold-out it fixed 12 and broke 0 against the decoder ($p = 0.0005$) and $+8/-0$ against the previous file ($p = 0.008$) (§rl-039–§rl-043).

Colloquial table (P4 series, v0.5.0). A colloquial bigram as a separate decoder term (P4) fixed 168 and broke 89 on discordtune at the decoder level ($+79$, $p < 0.0001$) but, once the full pipeline was re-scored (P4b), broke 92 rows there (128/92, $p = 0.018$) and 6 on the user-reported set—too many under the standard then in force. Moving the term into the re-rank fusion with the decoder's first choice as anchor and a soft gate (P4d) gave discordtune 74/43 (net $+31$, $p = 0.0053$, ratio 0.58), cvtune-clean 45/17 (lower bound $+0.46$ pp), user-reported 17/4, and the one-shot confirmation on chatjudge-2 passed: 108/39, net $+69$, $p = 1.1 \times 10^{-8}$, ratio 0.36, CER 2.85%→2.61%. The control arm that only raised α to 0.5 without the table was 61/38 ($+23$) on chatjudge-2 and 44/52 on discordtune; we attribute the gain to the table rather than to the stronger fusion (inferred from the control arm; not separately tested). A reading-channel term on top (P5) fixed 23 and broke 0 on discordtune ($p = 2.4 \times 10^{-7}$), 9/0 on cvtune-clean and 2/1 on user-reported; the one broken row was "長歪了" with 長 read ㄓㄤˇ, a reading the lexicon scores at the -6.8 floor. Both shipped in v0.5.0: discordtune chat $+88/-34$ ($p = 1.1 \times 10^{-6}$), lytune chat $+191/-27$, cvtune $+53/-16$, written wikitune $+39/-5$ ($p = 1.4 \times 10^{-7}$), with the hold-out written half $+8/-0$ ($p = 0.0078$) (§rl-046, §rl-048, §rl-053, §rl-054).

The SJ-LM-7 model generation (v0.6.0). Rebuilding the bigram with g0v and Forum A text and without `TWPhrases`, re-mapping the trigram, re-clustering, recounting the colloquial table and retraining the re-ranker proceeded in five registered stages. Stage 1 compared counting pack words or not (arms A, B) at pruning thresholds 2 and 6: all four passed conditions 1, 3 and 4 (B6: discordtune 4,366→4,534, $+201/-33$; user-reported chat/written 58/51→95/99) and all four failed condition 2 (dev302 written broke 6 rows: 祝你→助你, 嚐嚐看→常常看, 致病→治病, 再講→在講, …); the source counts are "在講" 2,340 in Forum A posts and replies vs 418 for "再講" (we infer, without a separate test, that Forum A brings the colloquial spellings of 在/再 into the same bigram; §rl-059). The six rows shipped as a recorded exception to condition 2 (§rl-061; [50]). Thresholds 3, 4, 5 and 8 left 6, 6, 5 and 6 of them broken, and re-selected written fusion cells 6–8 (§rl-060). Moore–Lewis selection of the new sources (keeping the best-scoring half, ML6) was worse than taking everything (B6): discordtune 4,534→4,523 (14/25, $p = 0.108$), and on the second-round internet-slang check set -35 written ($p < 0.0001$); B6 was kept. Recounting the colloquial table (C1) changed almost nothing against the old table (discordtune 4/0); a larger table (C2) exceeded the memory stop rule ($+43$ MiB) and hurt the conservative strength (1/11, $p = 0.006$). For the re-ranker, three recipes × three seeds were compared on chatclause by $U_2 = \text{fixed} - 2 \times \text{broken}$: the original recipe re-run (R0) lost to the shipped SJ-SR-6-14M in every seed (U_2 -34, -38, -45), the recipe with all new text (R1) won ($+24$, $+22$, $+11$), half the new text ($R\frac{1}{2}$) $+30$, $+8$, $+4$; the median R1 seed (5,211→5,275, 106/42, $p < 0.0001$) became 會意 SJ-SR-7-15M. Its own contribution on top of the new decoder (CAND vs B6+SR6, standard chat) was discordtune 4,558→4,620 (88/26), user-reported chat 24/1 and written 18/3; on wikitune-clean written it lost 1,634→1,616 (25/43, $p = 0.039$), which we read as the price of more colloquial training text (inference; the attribution was not measured). The final six-unit judgement (three strengths × two profiles) passed conditions 1, 3, 4 and the pack check in every unit, failed condition 2 on dev302 written in all three strengths (15/9, 18/12, 15/9; the known exception) and on typing76 written at the conservative strength (4/3, with the cell $(1, 1, 4)$ barely re-ranking, so the three rows are decoder errors). On the benchmark, v0.6.0 took user-reported chat from 33 to 70 and written from 29 to 61 of 80, discordtune chat 87.90%→93.69%, lytune chat $+295/-66$, and the hold-out talk half 115→124 ($p = 0.012$); the news

half 125→128 ($p = 0.25$). Memory in the Linux CLI rose 271→282 MiB (chat) and 376→380 MiB (written) (§rl-061–§rl-068). The stages and the final judging used sets of the tuning, development and regression layers and check sets, not a never-seen judging set; the public record contains no one-shot confirmation of this generation (Appendix E).

Reading term in the decoder (R1). The first offline design (R1, a reading term added to the decoder score before re-ranking, 12 cells) missed by one row each on dev302 written (0/1) and user-reported (2/3); two zero-cost checks found the sandhi reading ㄧˋ of 一 penalised by 1.39 and the 長=ㄓㄤˇ floor. R1b (sandhi exempt) and R1c (551 floor readings re-estimated from multi-character words) passed all four conditions; R1c's arm B-f fixed the user-reported row "長歪了" with everything else row-identical. The product design was then redone (R1d) to honour the rule that only semantic correction is strength-dependent: the term moved into the decoder path score for both profiles and with correction off—and R1d failed in chat with correction on (discordtune standard 60/51, cvtune-clean 15/38). R1d had also moved the colloquial term out of the fusion, and we attribute the loss at least partly to that (three things changed at once and were not measured apart); R1e, moving only the reading term, passed all eight settings (chat discordtune: off 17/0, conservative 16/0, standard 7/1, aggressive 6/1; written cvtune-clean 14/0, 12/1; dev302 0/0 everywhere). Re-measured on the v0.6.1 base (R1f) it failed condition 4 (the user-reported set; numbered condition 3 in the R1 registration) by one sentence, "搞完這波", twice (0/2): the lexicon scores 波=ㄆㄛ (−4.02) above the dictionary-standard ㄅㄛ (−4.72). A zero-cost check found 12 such characters; R1g exempted them and then had 0/0 on the same condition (net must be > 0), so it too "did not pass as registered". R1g went forward under a recorded exception: the only failing cell was that 0/0 and everything else passed, and R1g went to a one-shot confirmation on two never-seen sets under the shipping rule, with the confirmation slice locked before any number (§rl-086, §rl-087): wikijudge5 written standard $+15/-0$ ($p = 6.1 \times 10^{-5}$) and CE3 cjudge-1 chat standard 3/4 with a 90% lower bound of −0.089 pp (gate ≥ -1); both passed. Against v0.6.1 the shipped implementation is row-identical on dev302 and typing76, $+6/-0$ on user-reported chat ($p = 0.031$), costs ≤ 7.5 MiB and no measurable p95 in an idle window. A fresh verifier found one display-level regression on a sentence the contract itself had constructed: the term changed the anchor candidate, and the fusion then no longer chose the right string although neither candidate's scores changed (recorded, not fixed; §rl-081–§rl-085, §rl-088, §rl-089).

Re-ranker continuation training and new fusion cells (SJ-SR-8-15M). Two user-reported rows with 權限 were wrong because the re-ranker's training text had been converted by `TWPhrases` (權限→許可權). Retraining from scratch without the table (SR-TW) lost to the shipped re-ranker in all three seeds (U_2 −52, −44, −46 on chatclause) and failed conditions 1 and 4. A same-seed paired comparison (SR-TW-P) showed that the loss is no larger than retraining churn: any two re-rankers swap about 90–100 chatclause rows and 65–80 discordtune rows with net within ±12 ($p > 0.2$); not one of the 132+106 broken rows contained a `TWPhrases` key, while 26+7 fixed rows did (權限 12, 字體 4, …). Continuation training from the shipped weights (SR-TW-W: 3,000 steps at 1e-4, W1) cut the churn to 21–33 rows but still failed condition 1's ratio (12/9 = 0.75). Fixing the colloquial term's sentence-initial estimate (SR-TW-C, then the mild SR-TW-M) made all four 權限 rows correct, with W1+mild failing only condition 1 (13/11, ratio 0.85). Targeted fine-tuning on segments containing the target words (SR-TW-T) fixed 權限 but hurt the names check sets (1/5 written) and increased churn (38–54 rows). Several continuation recipes and seeds (SR-TW-V, 11 arms) did not reduce churn below the seed-noise floor (W1 against its own re-seeds swaps 39 and 33 rows; threshold 26.5), and only V4-s3 passed conditions 1–4 (16/9, ratio 0.56). It was confirmed

non-inferior on wikijudge7 (written standard 2,255→2,254, 5/6, lower bound −0.25 pp), and the TW0 tuning sets were rebuilt without `TWPhrases` for the standard's condition 3. A full-scan (SR-TW-F) found that under the old written cell $(0.5, 2, 0.5)$ the shipped and V4-s3 re-rankers are almost identical (3,356 vs 3,349 of 4,182 rows) and that both gain about the same from a stronger cell (3,420 vs 3,424 at $(0.8, 2, 0.25)$): the written gain is almost entirely the fusion cell, not the weights. Two confirmations then fixed the cells on fresh sets: FUSE-1 on wikijudge8 (2,844 rows), written $(0.8, 2, 0)$ vs $(0.5, 2, 0.5)$: 90/45, net +45, $p = 0.0001$, ratio 0.50; chat $(1.1, 2, 0)$ vs $(0.7, 2, 0)$: 43/17, net +26, $p = 0.0011$, ratio 0.40; FUSE-2 on wikijudge9 (2,827 rows), chat conservative $(1.3, 2, 1)$ vs $(0.7, 2, 1)$: 68/19, net +49, lower bound +1.20 pp; written conservative $(0.8, 2, 1.5)$ vs $(0.5, 2, 1)$: 14/0, lower bound +0.28 pp. The chat cells were selected on chat text (cvtune-clean-tw0 and discordtune) but, the chat judging set cjudge-1 having been used by R1g, confirmed on Wikipedia text run in the chat profile; their transfer to chat rests on condition 1 alone (below). The product (SJ-SR-8-15M, new cells, mild colloquial estimate) passed all four conditions at all three strengths against v0.6.1: standard discordtune $+29/-11$ (net +18, $p = 0.0064$), cvtune-clean-tw0 lower bound +0.10 pp, user-reported $+24/-6$, dev302 written $+9/-2$, typing76 unchanged; conservative discordtune net +35 ($p < 0.0001$), user-reported $+19/-0$; aggressive net +9 ($p = 0.12$), $+23/-6$. The twelve GCP runs for conditions 1 and 3 were byte-identical on a second machine. Two known limits were accepted rather than re-selecting the cells, because re-selection would have voided the FUSE-1 confirmation: user-reported row 179 ("權限加好了") remained wrong in the written standard and aggressive strengths, and on the 164-row re-selection learning check the written profile learns five rows fewer than v0.6.1 after one teaching (140 vs 145). The record's inference was that the learning term, on the n-gram scale, weighs less once α rose from 0.5 to 0.8; a correction of 2026-11-10 in the same record notes that in the groups concerned the teaching row was right at first display, so no learning record existed yet, and that scaling the learning term by 2.5 recovers only one row, so the cause is open (§rl-098–§rl-115). v0.7.0 then added the rerank-protect list to the mask (Section 3.5), which keeps the re-ranker from replacing 權限 and four other common words [50, 51].

Sentence-initial single-character learning (SINIT). Three registered versions of the learning rule were each tested on a project-written teach/rare/common case file and then on a fresh 6-group confirmation file. Revision 7: semantic groups net +19 on the case file but the confirmation flipped 3 "common" rows from right to wrong (limit 1): failed. Revision 8: common flips 0 on the case file; the confirmation flipped 2 (both "帶我出去玩", sharing three initial syllables with the taught "代我出席會議"): failed. Revision 9 (three prefix-key layers): case file 24 vs 12 rare rows right, 0 common flips; third confirmation file 13 vs 7, 1 common flip (limit 1), net +5: passed, byte-identical on two machines. With learning off, all eight sets were byte-identical to the baseline in every version (§rl-090–§rl-094).

Other shipped slices with measured selection effects. Neutral-tone tolerance (typing a neutral-tone word with its base tone) took a 300-word check set from 177 to 277 (chat) and 161 to 264 (written), breaking one user-reported row (accepted). Letting pack words keep their pack score (pack-oov) fixed 51 name groups at the decoder level, of which 8 were still overturned by the re-ranker—caught only because the check was re-run with the shipped flags. A Wikidata name pack of about 10,700 words in eight categories (Taiwanese people, TV dramas, games, bands, fictional characters, animated series and films, comics) at the very start changed no discordtune row and 1/0, 4/0 on the tuning sets, and nothing at all with only its three anime categories on; it was made an optional category pack rather than part of the default lexicon. The later 44,552-word anime

pack is on by default. Fuzzy-sound input (four pairs plus adjacent-key tolerance) shipped off by default: its penalty $\eta_{\text{sound}} = 2.5$ was the smallest keeping the clean-input broken rate $\leq 0.5\%$ in all four tuning cells, but it broke 2 rows of dev302 chat, both in the re-ranker rather than the decoder (Appendix D.4; §plan-010, §me-020, §rl-013, §rl-056).

D Complete list of negative results

This appendix lists every line of work recorded as "did not pass as registered", voided, retracted or stopped, with the recorded numbers. Table 6 classifies the main ones.

D.1 Judges and large models

- **N0, local LLM re-ranking.** Picking among unigram candidates with Gemma 4 E2B (M1) reached 91.4% on the development set at p95 848 ms on an RTX 3070; the gate was 95% and $\leq$ 300 ms. Reading-constrained character generation (M2) was worse, 82.5%, preferring fluent common words (試鏡布 for 拭鏡布). The ceiling was candidate coverage: the top 16 contained the answer 94.7% of the time (§rl-001).

- **N1, cloud judges.** Thirty-two provider/reasoning settings; the best was 92.7% (p95 3,146 ms), the best under one second 92.4% (p95 740 ms), all capped by candidate coverage. Reasoning off mostly cost no accuracy (§rl-002, §plan-006).

- **S5j, on-device foundation model as judge.** Apple's on-device model, asked to pick one of eight candidates, took discordtune from 83.5% to 75.4%; permuting the candidate order changed 49–55% of its choices; adding context made it worse. The same experiment showed +27 rows ($p = 0.0003$) on dev302, which is why dev302 is a challenge set and not a gate (§plan-013, §me-003).

- **S5k, local scorers.** No local model passed its pre-written gate; the best, Qwen3-1.7B 4-bit, was 87.2% vs 84.4% out-of-sample ($p = 0.081$), and scoring each candidate took p95 510–640 ms per row against a 300 ms limit (§plan-021).

D.2 Models and training objectives

- **S7b, a bigger re-ranker.** L (30.2 M parameters) was better on the public tuning sets but −30 rows on the author's chat and −97 against M; XL did not fit an 8 GB GPU. Removing the tuning-set rows that overlap training text (cvtune 507, wikitune 671) did not change the selected values or the verdict (§plan-014, §rl-044).

- **S7e, "one broken ≈ ten fixed".** Under that weight M was 70 fixed and 10 broken on the author's chat, a weighted utility $U_{10} = \text{fixed} - 10 \times \text{broken} = -30$, and L and XL scored $U_{10} = -138$ and -54; semantic correction shipped off by default in v0.4.0. Computing the ceiling afterwards showed the weight was unusable: even choosing values on the judging set itself, the best net was +17 to +23 of 4,091 rows, the same size as the noise from changing only platform and seed (two otherwise identical runs differed by 32 rows; the noise line used in S7f was −21). The weight was replaced by 1:1 scoring with a broken-rate cap (§plan-014, §me-011).

- **S7f, retraining on human colloquial text.** A1 added 20 million characters of open-licence colloquial text; A2 added the same plus 2.5 million synthetic characters from each

of two open models. Under the 1:10 weight A1 scored $U_{10} = -126$ and A2 -107 (noise line -21); before a data-cleaning bug was fixed the numbers were -46 and -171 with the same verdict. Re-scored under 1:1 on 2026-10-18 (E-A), A2's estimated net on chatjudge-2 was +16, below the minimum detectable effect +32, and the development-layer broken-rate upper bound exceeded 0.5%; with three seeds per arm the estimate stayed below the MDE. The colloquial text nevertheless entered SJ-LM-7 and SJ-SR-7, where it did help (§plan-014, §ip-004, §ip-005).

- **rerank-size, L and XL on the trigram decoder.** L188 (written) exceeded the 0.5% upper bound in all five cells (lowest 0.574%); XL's selected cells reached 0.430% written and 0.400% chat; the registered Holm test could not be passed at the effect sizes seen; the slice was later reopened under the current standard but not re-run (§rl-045, §rl-052).
- **L0, listwise objective on M.** Standalone pick accuracy fell to 19.1% and discordtune net against the shipped M was -38; both gates failed (§ip-006).
- **P1/P2 and the distillation line.** 0c re-scored on current candidates was +105 net on discordtune but its development-layer upper bound was 0.61% ($> 0.5\%$); a distilled M-size student (P2) was -2 (upper bound 0.39%); with readings as input (B) -2; with Wikipedia rows removed and a zero-training diagnostic (C) -35; an L-size student +7; position-level distillation reached 55.3% agreement against a 72% gate and -41. The student line was closed under its pause clause (§ip-006).
- **P3 and the long-context series.** Lengthening the student's context from 2 to 63 characters: -39. Feeding the shipped M 64 characters without retraining: discordtune -1, tune -7, not significant. The CTX64 series (retrain, continue, continue with rehearsal) then failed its "three seeds all $U_2 > 0$" direction rule; the measuring-stick check CTX64-M showed that the shipped cell had been selected for one seed: the same recipe re-seeded scores U_2 -50 and -63 against it on the selection set, so the rule could not tell a retrained model from a re-seed; after per-arm cell selection on a held-out half the challengers were U_2 0, -9, -32 against seed noise -6, -17—no direction. Keeping trailing punctuation in the context column made things worse (U_2 -26, -18, -17). The line is paused (§ip-006, §rl-080, §me-023).
- **SR-TW from scratch and SR-TW-T.** See Appendix C: retraining from scratch lost by no more than same-recipe retraining churn; targeted fine-tuning fixed the target rows and hurt the names check sets.
- **E-B, reading-prefix input format.** One-seed pilot: +32 net on the tuning layer vs +91 for the control; stopped at the pilot (§ip-005).
- **E-F, learned gate.** A cost-sensitive classifier replacing (α, τ, δ) with numeric features beat the same-rule baseline by net 13 but not the shipped values, and its upper bound exceeded 0.5% (§ip-005).

D.3 Decoder terms and data

- **Synthetic confusion sentences** (round 2, 2026-10-03): 21,611 generated sentences targeted at confusable pairs regressed both tuning and chat sets (§rl-006).
- **Mixing corpus frequency into the prior** (mixing weight 0.3 or 0.5; unrelated to the Kneser–Ney β_p) lost over 100 chat clauses: the corpus is mostly Wikipedia, so the prior absorbed a written-register bias (§rl-011, §rl-012).

- **Wikipedia Taiwan-variant conversion (W, W2).** Not vetoed by gates; hold-out chat −2, written +1; a user report turned "的市佔" into "的士站"; not shipped (§plan-020).

- **P1a, a word-sense classifier for 在/再.** Training only ㄗㄞˋ triggered the stop rule: cvtune chat 再>在 errors 11→10 (halving required); the classifier's per-position accuracy was below the n-gram; the class prior from capped sampling was distorted and the fusion had mixed natural and base-10 logarithms until review (§plan-011, §me-010).

- **s4-bayes, Dirichlet posterior for learning.** None of 40 settings met the criteria; the mirror-teaching rate was below the current rule in every setting (0–3/8 vs 7/8; §plan-022).

- **derisk-1015, E1 trigram size.** Of trigram tables pruned at 15, 3, 2 and none, only the unpruned table (71,858,873 entries, 1,133.3 MB) passed the registered judgement at its selected values; re-judged at the shipped λ none passed ($\alpha = 0.0083$), and we read the gains as shrinking monotonically with pruning (inferred from six table sizes). The 7.7 M-entry table nonetheless shipped after a separate re-judgement at the shipped λ with Kneser–Ney off (above). Lossless compression would save 7.8 MB of install size for the bigram; 4-bit quantisation was significantly worse (§rl-020, §rl-024, §rl-027).

- **Re-tuning $\lambda \times \beta$ with Kneser–Ney**: the tuning set pushed λ to the grid edge and discord-tune lost significantly (−39 chat, −43 written, $p \leq 0.0001$; §rl-022).

- **E4(b), context after digits/English**: main judgement +18 ($p = 0.0198$) but wikitune 2/12 ($p = 0.013$) and 2/11; failed the gate. **E4(a)**, not resetting context at mid-sentence punctuation: −1 and −8. **E3**, whole-string initial abbreviation: top-1 ratio 0.789/0.808 against a 0.9 gate, bottleneck in ranking (oracle@64 still 289/302), 148 ms per row (§rl-023, §rl-025, §rl-026).

- **Reading-aware counting (B)**: counting bigrams per reading cost 1,457 rows on lytune chat (958 of them 嘛→麻, 493 啦→拉): the rule-based reading annotation was wrong in 64 of 157 sampled positions (40.8%; adverbial 地 tagged ㄉㄧˋ, auxiliary 得 tagged neutral), against a 10% stop rule. **A2**, demoting first-tone readings of sentence-final particles: lytune −213/−508 at $\delta = 0.25/0.5$. A1, a 44-character neutral-tone demotion table, broke nothing and shipped (§rl-038).

- **E-D, candidate coverage.** Widening per-reading candidates from 12 to 24/48/1000 raised chat oracle@64 by at most 0.03 pp. **E-C**, joint tuning of decoder weights and fusion: the best chat cell was 5 rows above the shipped point; the best written cell was the shipped point (§ip-005).

- **BRIDGE-1/2, a one-word-skip class bridge.** Positive on chat tuning cells (cvtune +8 to +17) but negative on written where a trigram already exists, with broken 80 vs fixed 107; chat-only on the filtered chatclause, which this line had not seen (BRIDGE-2): 0/2, and the arm registered had not been the one measured before (a documented registration error; §rl-070, §rl-077).

- **LEXGAP-0/1, suffix compounds (X氏, X室).** Adding 694 words to the overlay changed not one row of ten set/profile combinations because overlay words carry the floor score minus 1.0; rebuilding the bigram with them in the counting vocabulary (691 words, +2,672 bigrams) fixed exactly one row (桑拿室). The line was closed, with the word list carried into the next rebuild (§rl-095, §rl-096).

- **Moore–Lewis selection** of Forum A and g0v text (above): worse than taking everything (§rl-061).
- **HPLT and Wikisource as long-context evaluation text.** Three 300-row samples of HPLT 3.0 Traditional forum/blog text had 12.7%, 5.3% and 6.7% of rows unusable, mainly SEO-inserted real words; Wikisource public-domain prose was 37% simplified originals, and after those were filtered out the second proofreading still had to correct 13.8% of the rows; both abandoned (§me-016).
- **OpenCC conversion as a root cause.** `TWPhrases` had been rewriting 127 lexicon words (71 frequent: 支持, 設備, 程序, 權限, …), 47 of them with same-reading same-length rivals; on 94 constructed sentences the shipped re-ranker broke 7 of 49 decoder-correct rows (chat) and 7 of 46 (written) where the retrained W1 broke 1 and 0. Separately, `STCharacters` converted 湿 only to 溼 while MediaWiki uses 濕; in the first 2,000 articles 381 occurrences of 溼 came from converted simplified sentences and 18 from traditional ones (for 濕: 29 and 135), against Forum A usage of 濕 1,418 vs 溼 106. Half the Wikipedia sentences (49.3%) are genuinely simplified originals; 20,436 of the sentences flagged simplified contained no simplified-only character. A conversion repair slice (TW-CONV) built a 29,918-row correction table by LLM-assisted proofreading with blind spot checks (gap1: 64 of 69 due changes made, 0 of 41 keep-rows broken), and the next rebuild will use it (§rl-107, §rl-117, §rl-118).

D.4 Fusion, decisions and failed confirmations

- **rerank-formal standard cell**: primary judgement passed, broken-rate gate failed (1.72% vs 0.50% on tuning data)—the case that changed the gate to an upper bound (§rl-039, §rl-040).
- **P4b, P4c**: see above; P4c's chatclause-selected cell had ratio 0.45 on the selection set and 0.60 on discordtune, the same upward drift seen in P4b and P4d (0.33→0.58; §rl-050, §me-017).
- **DECIDE-0/1, relaxing written fusion.** Exploration found all eight relaxed written cells passing the direction rule, best $\alpha = 0.75, \delta = 0$ (net +108 over three sets, ratio 0.42); the confirmation on never-seen wikijudge4 (2,820 rows) gave $+67/-48$, net +19, $p = 0.0928$, ratio 0.72: failed on two of three criteria. The later FUSE-1 confirmation of $(0.8, 2, 0)$ with the new weights passed on wikijudge8 (ratio 0.50) (§rl-075, §rl-076).
- **Fuzzy input penalty** η_{sound} (above): 0/2 on dev302 chat; both rows lost in the re-ranker after the decoder had already demoted the fuzzy word.
- **R1, R1d, R1f, R1g** (Appendix C): three misses by one row and one 0/0 before the exception; R1g then confirmed.
- **SINIT revisions 7 and 8**: 3 and 2 common-row flips against a limit of 1.
- **SR-TW, -W, -C, -M, -T, -V**: eleven arms in the last exploration, none passing all gates; V4-s3 the only one passing conditions 1–4.
- **LOSS-0/L5-0 diagnostics** (exploration): among homophone errors still wrong after the full pipeline, 28–64% are rows where the re-ranker preferred the answer but the gate kept the decoder (layer 4), and 34–52% where the re-ranker also preferred the wrong string (layer 5); in the written profile three to four tenths of layer 4 are stopped by δ and almost none by τ. The upper bound of rows a stronger re-ranker could rescue in two chat tuning sets is

246, but about half the sampled rows were judged undecidable, so the real number is perhaps half that (inference). A third check in this slice did not pass its registered score-comparison tolerance (a $\log_{10}$ difference of at most 10^{-6}; the largest observed was 1.6×10^{-5}) and its table is recorded but not interpreted (§rl-073, §rl-074, §rl-078, §rl-079).

D.5 Product changes reverted or deferred

- Treating an impossible-Zhuyin key sequence as English by key order shipped on by default and was reverted the same day after too many false triggers in fast typing (later keys of a syllable arriving first); the rule returned with a "reversed key order is not English" guard, and two related switches default to off from v0.7.0 (§me-022; [50]).
- "Keep other characters fixed after a re-selection" was paused after its fifth review found the "continue typing after re-selecting" flow worse (whole-sentence correct 1500→1438 of 2165); it was taken up again later and shipped in v0.5.0 (§plan-019; [50]).
- A Bayesian learning rule, a global learning layer and an abbreviation mode remain off.

E Engineering controls and process deviations

E.1 Engineering controls

All arms of one comparison run on one CPU type: the same 135 rows on an Intel desktop and on an AMD EPYC 9B14 cloud VM gave identical selections but small-model scores differing by up to 7.1×10^{-6} per row (§me-015). Since 2026-10-29 the confirmation and shipping-standard runs are taken on two machines and their row results compared byte for byte (small-model scores differ at the 10^{-5} level and are compared within a tolerance); the hold-out is run once, by one agent. Hashes of every model, lexicon, pack and evaluation file are checked before each run and the CLI prints a configuration tag that must match the expected one. Code review and acceptance are done by independent agents. The hold-out is run only by such an agent, only before a release tag, and only its summary line is returned.

E.2 Process deviations

The process broke its own rules, or did not apply them, in the following places; we list them because they bound what the results can claim. (1) The shipping standard was relaxed twice after seeing numbers (2026-10-20); the slices that passed only under the relaxed rule were re-confirmed on chatjudge-2 (§rl-049, §rl-053). (2) P5 (the reading channel) shipped without a never-seen confirmation after its advisor had seen two of the regression sets; no unused chat judging set existed at the time (chatjudge-2 had been used by P4d, and at most one judging set is drawn per month), the proposal to wait for a chatjudge-3 drawn in November was not adopted, and P5 shipped together with the colloquial table in v0.5.0 (§rl-054, §me-018). (3) R1g went to confirmation by exception after a 0/0 on condition 4 (§rl-086). (4) The P4d confirmation was written as a README section rather than a one-page registration with an MDE (§me-017). (5) DECIDE-1's second source was run on the Mac against the "no heavy compute on the Mac" rule (§rl-076). (6) In the trigram slice B2 the stop condition (manual words 2/15 wrong against a limit of 1) was reported after B3 had been allowed to finish; the two rows were then recorded as a known issue, the B2 condition relaxed, and the slice continued (§rl-034, §rl-036). (7) model-v5's hold-out was run after the release that contained it (§me-009). (8) The L5-0 check 3 tolerance in code did not match its registration

(§rl-079). (9) BRIDGE-2's registered arm was not the arm BRIDGE-1 had measured (§rl-077). (10) Several early "closed" labels written by the implementing agent were reclassified on 2026-10-20 as "did not pass as registered", because under the rule adopted that day a failed registration is not a closure (§rl-052). (11) The model generation of v0.6.0, the largest step of Table 2 (the bigram rebuilt with forum and chat-channel text, the class, Kneser–Ney, trigram and colloquial files built with it, and the re-ranker SJ-SR-7-15M), shipped on its final judging against the shipping standard, on sets of the tuning, development and regression layers that had been seen while its arms and values were chosen; the public record contains no one-shot confirmation of it (§rl-059–§rl-068; [50]). What this paper has for it on data not used for selection is the monitoring hold-out (Table 3). (12) Table 1 lists every one-shot confirmation that was run, so the other shipped changes that are not in it had none either: in v0.7.0, the sentence-initial estimate of the colloquial term (for the reason given at the start of Section 3), the rule for tied readings and the rerank-protect list; and the lexicon layers, the packs and the input tolerances (Appendix A and the last paragraph of Appendix C), which, where the record gives numbers, were measured on check sets and on sets already seen.

F A day-one snapshot of existing input methods

On the first day of the project the same sentences were typed into three existing IMEs with a key-sending automation tool (§me-005, §me-006): the macOS built-in Zhuyin, McBopomofo (小麥注音, MIT, phrase data from libtabe) [37] and a commercial input method. On 23 written sentences they made 3, 10 and 8 sentence errors; on 53 colloquial sentences 8, 19 and 22; on 845 private chat clauses 100, 199 and 237; on 294 government press-release sentences 64, 85 and 94. The built-in IME was best throughout, and it was found to learn during the test: of 2,647 phrases it learned, 847 appeared in its own test output (§rl-004).

These numbers are a single-day snapshot (2026-10-03) and carry four limits. The products keep changing, and the snapshot was not repeated. The built-in IME carried its learning state. The readings came from an early version of the project's reading tool, so one mis-read character fails all three IMEs alike. And the sets are the project's own: the 76 written and colloquial sentences are the typing76 regression set of Table 2, on which Shanjie was later tuned, and the 294 press-release sentences became training text on 2026-10-29 and are no longer a test set. The snapshot motivated the project; it is not a comparison with Shanjie, and no Shanjie number is set beside it.

Other tools examined in the development records: vChewing (唯音) for its mixed Chinese–English handling [53]; the closed-source ZingIME (a McBopomofo fork with a fine-tuned local Qwen2.5-0.5B); the Python research core zaoseq-bopomofo (which drops heteronym readings not attested in known words and reports a "broken $\leq 0.5\times$ fixed" acceptance rule); and Jevboard, a Windows tool that asks the Jev judgement model to pick characters (§rl-057, §plan-015, §plan-016).

G Global learning layer sweep

Table 10: Global learning bonus ε with the class term on (2026-10-08): mirror-learning rate and polluted rows. Without the class term, $\varepsilon = 0.5$ gave 8/9 learned and 0 polluted.

$\varepsilon_{\text{global}}$	learned	polluted rows
0	0/8	0
0.1	3/8	1
0.5	4/8	3
1.0	6/8	4
1.5	7/8	7
2.0	8/8	9
6.0	8/8	29

H Model files and codes

The fusion triples of the re-ranker shipped in v0.6.1 (SJ-SR-7-15M) were chat $(0.7, 2, 1)$, $(0.7, 2, 0)$, $(1.0, 2, 0.1)$ and written $(0.5, 2, 1)$, $(0.5, 2, 0.5)$, $(0.5, 2, 0.5)$ for the conservative, standard and aggressive strengths; those of v0.7.0 (SJ-SR-8-15M) are in Section 3.5.

Table 11: Model files of v0.7.0, with the Chinese function name and the code of each. v0.6.1 shipped the same files except the re-ranker, which was SJ-SR-7-15M (the same size); SJ-SR-8-15M replaced it on the main branch on 2026-11-06.

Chinese name	Code	File	Size measure
通語	SJ-LM-7-9.7M	bigram.sjlm	9,709,510 bigrams, 396,927 words
辨類	SJ-WC-6-512C	classes.sjc	512 classes
承接	SJ-KN-2-397K	kn.sjkn	396,927 words
連文	SJ-CX-2-7.7M	trigram.sjt3	7,700,964 trigrams (written)
連文	SJ-CX-2-950K	trigram-chat.sjt3	950,000 trigrams (chat)
會意	SJ-SR-8-15M	tinylm.sjtl	14,646,912 parameters
常談	SJ-CL-2-69K	colloquial.sjlm	69,457 words